

V | MANUAL

01 MANUAL:OBJECTIVE

Objetivo

¿Qué problema que resuelve?

Una búsqueda laboral sin estructura produce cuatro fallas operativas concretas:

- Oportunidades de alta señal desaparecen antes de ser procesadas.
 - Tiempo consumido en vacantes irrelevantes que no cumplen criterios mínimos.
 - Aplicaciones enviadas sin datos de fit — sin score, sin análisis de keywords, sin estrategia de CV.
 - Sin trazabilidad: qué se aplicó, cuándo, qué sigue.

¿Qué hace diferente?

- Convierte la búsqueda laboral en un pipeline con contratos de procesamiento definidos.
 - Filtra antes de evaluar:
 - Links muertos → Score 0, Status Expirada.
 - Roles sin componente visual → Gate BLOCKED.
 - Empresas en lista negra → rechazadas en Discovery.

Las empresas excluidas permanentemente (Hard Blocks) y las reglas de deduplicación están documentadas de forma completa en MANUAL:DATA-MANAGEMENT

- Verifica antes de crear:
 - Cada URL pasa un chequeo de enlace (lo que el sistema llama internamente "URL_GATE") antes de cualquier cálculo de fit.
 - Si el link no funciona, la vacante no entra al pipeline activo.

El mecanismo completo de este chequeo y los pasos que le siguen están explicados en MANUAL:HOW-IT-WORKS.

- Centraliza en un solo lugar:
 - Notion es la fuente única de verdad — vacantes, aplicaciones, scores, seguimiento
 - Calcula con lógica determinista:
 - Score 0-100 calculado por Python.
 - La decisión de postulación se toma con datos, no con estimaciones.

¿Cuáles son los KPIs del sistema?

Métricas calculables directamente desde el Tracker, sin requerir instrumentación adicional:

- Throughput semanal — vacantes nuevas ingresadas por ciclo de feed_processor.py.
 - Tasa de Gate CREATE — % de vacantes con Gate_Decision=CREATE sobre el total procesado.
 - Tasa de recuperación RT-1 — % de vacantes BLOCKED que alcanzan

PATCHED → CREATE vía Dashboard.

- Distribución por Positioning Mode — vacantes activas por N1–N4 (campo Positioning_Mode).
- Score promedio de vacantes CREATE — calidad promedio del pipeline activo.
- Nota: el sistema no mantiene checkpoints de tiempo entre estados (ej. tiempo de ingesta a READY_TO_APPLY) — cualquier métrica de latencia queda fuera de esta lista hasta que exista un campo de timestamp dedicado.

¿Qué no hace el sistema?

- No busca cualquier empleo — solo roles visuales en sectores lujo, premium, cool DNA y agencias de experiencia.
 - No genera volumen masivo — calidad de señal sobre cantidad de resultados.
 - No aplica automáticamente — la decisión de postulación es siempre humana.
 - No adivina campos faltantes — si falta información, el campo queda pendiente y el sistema lo reporta.

¿Qué tipo de vacantes busca?

- Perfil Senior en:
 - Visual Merchandising
 - Brand Environment
 - Store Design
 - Retail Experience.
 - Geografía:
 - CDMX
 - LATAM.
 - Sectores target:
 - Lujo (LVMH, Kering, Richemont),
 - Retail Premium (Nike, Apple, Inditex)
 - Cool DNA (Gentle Monster, Ben & Frank)
 - Agencias de experiencia.

——02 MANUAL:HOW-IT-WORKS

¿Cómo funciona?

El Pipeline

- El Pipeline opera secuencialmente.
 - Cada paso tiene un responsable (una capa del sistema: L1, L2, L3, Python, o el operador humano) y un output definido antes de pasar al siguiente paso.

El detalle operativo día por día de este flujo está en MANUAL:WEEKLY-FLOW aquí se explica la lógica que sostiene ese flujo.

Layers

El sistema se organiza en capas con responsabilidades separadas:

- L1 - Active Recon: Búsqueda activa vía prompts ejecutados en motores de búsqueda (Perplexity, Comet).
 - L2 - Strategic Search: Búsqueda activa complementaria vía Gemini, Grok, you.com.
 - L3 - Passive Intake: Lectura automática de correos etiquetados en Gmail.
 - L4 - Documentation: mantiene sincronizados en background el repositorio de código (git) y los documentos fundacionales del sistema entre Notion y disco local.

Responsables

- Python (Pipeline / Runtime): normaliza, deduplica, calcula Score y Gate_Decision, y expone comandos de consulta.
 - Agente IA (Claude): opera el Ciclo de Sesión (Open/Close), ejecuta CV-A/ CV-B en el ciclo semanal (Miércoles), y mantiene documentos fundacionales y trackers vía parches — siempre bajo DRY RUN + APROBAR_WRITE, nunca escribe sin autorización explícita del operador.
 - El operador: Decide qué se postula, resuelve bloqueos recuperables vía Dashboard (Proponer Patch → Validar → Aceptar, sobre campos Class A del Tracker), autoriza escrituras y aprueba entregables.

Este reparto de trabajo se ve en acción completa en MANUAL:WEEKLY-FLOW

Gate Decisions

Este es uno de los conceptos que más se usa a lo largo de todo el ciclo operativo, así que conviene fijarlo aquí, antes de encontrarlo en Lunes, Martes o Miércoles sin previo aviso.

El sistema evalúa cada vacante nueva en tres pasos, siempre en este orden:

1. Link check — si la URL de la vacante no carga (404, 403, dominio caído, redirección rota), la vacante se archiva automáticamente con Score 0 y Status "Archivar". No se calcula nada más sobre ella: un link muerto no tiene fit que evaluar. Esto es lo que el sistema llama internamente el "URL_GATE" — el primer filtro que cualquier vacante debe pasar antes de que Python invierta cómputo en analizarla. Para vacantes en agregadores conocidos, este chequeo ya no es una excepción sin verificar — se ejecuta con un timeout corto para evitar cuelgues, pero registra honestamente si no pudo confirmarse.
2. Score (0–100) — si la URL funciona, Python calcula un puntaje numérico según qué tan bien encaja el rol con el perfil objetivo (Keywords de Visual Merchandising, sSctor, Seniority, Geografía). Este cálculo es determinista: mismos datos de entrada, mismo Score de salida, siempre.
3. Dónde aterriza, según el Score obtenido:
4. | SCORE | NEXT ACTION |
5. | --- | --- |
6. | 60 o más | READY-TO-APPLY: Tu bandeja de trabajo diaria — de aquí

sale todo lo que trabajas en CV Optimization. |

7. | 40–59 | REVIEW_NEEDED: Zona gris: el sistema no descarta la vacante, pero tampoco la prioriza — la decisión de trabajarla o no es tuya. |

8. | Menos de 40 | ARCHIVE¹ |

- Excepción a los tres pasos: si la vacante llegó por contacto directo (Inbound, Referencia o Networking), se salta este proceso completo y entra directo como CREATE — un contacto humano pesa más que el algoritmo, porque la señal de calidad ya viene validada por una persona real, no por texto de un JD.
 - Este mismo mecanismo de Gate es el que determina si una vacante queda en estado BLOCKED cuando algo en sus datos de entrada (Class A: URL, JD, Source_Type, Prioridad) es inconsistente².

¹ ¿Por qué esto no es un error que debas corregir? ver MANUAL:FAILURE-PHILOSOPHY.

² Para caso específico y cómo recuperarlo a través del Dashboard ver MANUAL:TUESDAY.**

Soft vs Hard Blocks

El sistema aplica dos capas de exclusión para garantizar la calidad de la señal — ambas se explican con su lista completa y mecánica de recuperación en MANUAL:DATA-MANAGEMENT, pero conviene entender la diferencia conceptual desde ahora, porque ambos términos aparecen constantemente en el flujo semanal:

- Hard Blocks: Empresas o roles que nunca entrarán al sistema (ej. L'Oréal, Levi's). Se filtran en el origen, antes de que la vacante exista siquiera como registro en Notion, y no son recuperables bajo ninguna circunstancia.
 - Soft Blocks: Vacantes bloqueadas por inconsistencias en datos Class A (URL rota, JD parcial) o por Score insuficiente. A diferencia de los Hard Blocks, estas sí son recuperables — corrigiendo el dato erróneo a través del Dashboard MANUAL:WEEKLY-FLOW-002.

———**03 MANUAL:FAILURE-PHILOSOPHY**Filosofía de Fallo para Operadores Base KERNEL:FAIL-PHILOSOPHY

Antes de entrar a Setup y al ciclo operativo, es necesario internalizar esto, porque vas a encontrarlo constantemente desde el primer Lunes que operes el sistema: Un “fallo” del sistema no es un bug — es el filtro operando correctamente. Los siguientes resultados son señales normales de funcionamiento, no errores que debas reparar manualmente ni forzar a que Claude “arregle”:

RESULTADO	QUE SIGNIFICA	QUE NO HACER	QUE SI HACER
-----------	---------------	--------------	--------------

URL dead / link roto	La vacante expiró — es normal del mercado laboral, las publicaciones caducan.	No repares manualmente el link ni intentes “revivir” la vacante.	Déjala archivada. Si te parece un error de captura (ej. la URL se guardó mal, no que la vacante haya expirado), eso sí es corregible — ver MANUAL:WEEKLY-FLOW-002, Dashboard.
Score = 0	El fit es débil, o el link estaba muerto (ver MANUAL:HOW-IT-WORKS, Gate Decisions, paso 1).	No subas el score a mano — es un campo Class B, calculado por Python, no editable directamente.	Si sospechas que el cálculo está mal por un dato de entrada erróneo (URL, JD), corrige el dato de entrada, no el resultado — ver MANUAL:WEEKLY-FLOW-002.
Gate = BLOCKED	Los criterios Class A no se cumplieron (URL rota, JD parcial, fuente no reconocida).	No lo fuerces a CREATE manualmente en Notion.	Si es recuperable, usa el Dashboard (MANUAL:WEEKLY-FLOW-002) para corregir el dato de origen y dejar que el pipeline recalcule.

Ready-to-Apply vacío	No hay oportunidades válidas esta semana — puede pasar, especialmente en semanas de baja actividad del mercado.	No fuerces un CREATE artificial para “llenar” la bandeja.	Espera al siguiente ciclo de discovery (Lunes), o revisa si el Prompt de búsqueda necesita ajuste (ver MANUAL:HEALTHCHECK, Red Flags).
JSON vacío en el Feed de discovery	La búsqueda no encontró resultados relevantes esa ejecución.	No amplíes los criterios de búsqueda sin análisis previo — podrías bajar la calidad de la señal general.	Revisa el Viernes de Analytics (MANUAL:WEEKLY-FLOW-005) antes de decidir si el prompt necesita ajuste.
Ante cualquiera de estos casos, el sistema reporta el estado y espera tu instrucción dentro del flujo normal del Pipeline — no requiere, ni acepta bien, intervenciones manuales que intenten “corregir” el resultado en sí mismo en vez de corregir el dato de entrada que lo produjo.			

——04 MANUAL:SETUP

Setup

Esta sección se ejecuta una sola vez, al instalar el sistema por primera vez (o al

reinstalarlo desde cero). Si el sistema ya está instalado y solo llevas varios días sin usarlo, lo que necesitas es MANUAL:COLD-START, no este capítulo completo.

Prerrequisitos

- Cuenta de Notion con base de datos VANTAGE TRACKER activa.
 - Python 3.8+ instalado en Mac.
 - Acceso a Claude.
 - Cuenta de Perplexity con modo Deep Research activo.
 - Acceso a Gemini con modo Deep Research o Search activo.
 - Acceso a You.com con modo Research o Agent activo.
 - Acceso a Grok con modo DeepSearch o Think activo.

Paso 1

Verificar Notion

- READY-TO-APPLY: Espacio de trabajo diario (Score ≥ 60 , Gate_Decision=CREATE).
 - REVIEW_NEEDED: Vacantes en rango Score 40–59 (Gate_Decision=REVIEW_NEEDED, requieren revisión manual).
 - BLOCKED: Score < 40 o fail de scope/fetch (Gate_Decision=BLOCKED).
 - ARCHIVE: Score 0 o Status Expirada.
 - ALL: Administración general.

Paso 2

Instalar Entorno Python

```
cd ~/Documents/03 Projects/VANTAGE/Layer_1
source .venv/bin/activate
# (el entorno ya existe; solo actívalo)
```

En Terminal, verifica la instalación, debe mostrar 3.8 o superior.

```
python3 --version
```

Paso 3

Bootloader con Claude

Ya no es necesario realizar copy-paste manual del System Prompt maestro en cada actualización, en su lugar:

1. Las instrucciones activas deben residir exclusivamente en las Project Instructions de la plataforma. Encontrarás la referencia documental en SP:BOOTLOADER. Este es el proceso para su configuración:
 2. Settings → Project → Project Instructions en la UI de Claude.
 3. Inicia un nuevo chat. El Agente realizará un fetch automático del Bootloader desde Notion.
 4. El Agente responde con "VANTAGE: SISTEMA SINCRONIZADO" (sin número de versión fijo — ver SP:BOOTLOADER) antes de enviar peticiones.
 5. Nota: este setup de Claude es de una sola vez por proyecto — no se repite en cada sesión de trabajo. Lo que sí se repite en cada sesión es el

Ciclo de Sesión completo, explicado en MANUAL:SESSION-CYCLE.

Paso 4

Verificar Archivos del Sistema y Permisos de Ejecución

Confirma que los archivos del sistema existen en tu Mac en las rutas esperadas (Layer_1, Layer_3, Layer_4, Dashboard). Si reinstalas o mueves archivos, verifica permisos de ejecución:

```
chmod +x $LAYER_1_DIR/layer_1_pipeline.sh
chmod +x $LAYER_1_DIR/wrappers/layer_1_wrapper.sh
chmod +x $LAYER_3_DIR/wrappers/layer_3_mail.sh
chmod +x $DASHBOARD_DIR/wrappers/dashboard_start.sh
```

Paso 5

Test Inicial del Pipeline

```
~/vantage_pipeline.sh tracker
```

Output esperado:

```
= VANTAGE PIPELINE STATUS =
Ready-to-Apply: [N] vacantes
REVIEW_NEEDED: [N] vacantes
...
```

Si falla: verifica que ~/vantage_notion_audit/.env existe y contiene tu token de Notion.

Paso 6

Verificar Runtime

El Runtime (explicado en detalle en MANUAL:RUNTIME) es el motor de lectura del sistema — permite consultar el estado de Notion desde Terminal sin abrir el navegador. Antes de operar, verifica que su índice interno (el "Entity Index", el catálogo de todas las entidades que el Runtime sabe interpretar) esté cargado:

```
python vantage.py status
```

Resultado esperado: Status: READY (4,200+ blocks indexed).

Paso 7

Verificar Sync Documental

```
cd ~/Documents/03 Projects/VANTAGE/Layer_4/scripts
source ../../Layer_1/.venv/bin/activate
python vsync_doc.py --dry-run
```

Output esperado: 6 documentos listados con diff por documento, sin errores.

Si falla: verificar que layer_1.env exista y que el token no tenga un salto de línea (\n) embebido por error de copy-paste.

05 MANUAL:COLD-START

Arranque Frío

Usar cuando el sistema no ha sido operado por más de 5 días. A diferencia de

MANUAL:SETUP, aquí no estás instalando nada nuevo — estás confirmando que todo lo que ya instalaste sigue funcionando después de un periodo de inactividad, antes de confiar en que el primer comando que corras te va a dar un resultado correcto.

```
cd ~/Documents/03\ Projects/VANTAGE/Layer_1/scripts && \  
source ../.venv/bin/activate && \  
python3 --version && \  
cat ../.env | grep NOTION_TOKEN && \  
python3 vantage.py status && \  
python3 vantage.py sync && \  
vl3 && \  
cat ~/.vantage/l3_heartbeat.json && \  
python3 vantage.py ask "show active roles" && \  
python3 vantage.py ask "find candidates"
```

Explicación por comando

1. `cd ...` — te posiciona en `Layer_1/scripts`, donde viven `vantage.py` y todos los scripts del runtime.
2. `source ../.venv/bin/activate` — activa el virtualenv correcto (un nivel arriba de scripts), asegurando que uses las dependencias instaladas del proyecto y no el Python global.
3. `python3 --version` — confirma que el intérprete activo cumple el mínimo (3.8+); no bloquea el resto si falla, solo informa.
4. `cat ../.env | grep NOTION_TOKEN` — verifica que el token de Notion esté presente en el `.env`; no valida si está vigente, solo que la variable existe.
5. `python3 vantage.py status` — healthcheck de solo lectura: lee `entity_index_v2.json` y las métricas acumuladas de `notion_utils`, sin tocar la API.
6. `python3 vantage.py sync` — regenera `entity_index_v2.json`, `graph_v2.json` y `backlinks_v2.json` consultando Notion en vivo; sí escribe.
7. `vl3` — corre `layer_3_mail.py` una vez, manualmente (asumiendo que `vl3` es tu alias configurado para ese script).
8. `cat ~/.vantage/l3_heartbeat.json` — confirma que L3 corrió y dejó su heartbeat; si el archivo no existe o el timestamp es viejo, L3 falló silenciosamente.
9. `python3 vantage.py ask "show active roles"` — smoke test end-to-end contra el Tracker real vía `resolver_layer_v1` — este es el que hoy falló por el archivo borrado en el commit 29fc7f0; seguirá fallando igual hasta que restauremos ese archivo.
10. `python3 vantage.py ask "find candidates"` — mismo smoke test, distinto intent; mismo resultado esperado (falla) por la misma causa.
11. Nota: con el `&&` encadenado, si `sync` o cualquier paso previo falla con

código de salida distinto de cero, los comandos posteriores no corren. Si quieres que seguridad no se detenga a medio camino (por ejemplo, si vl3 no es un alias válido en tu shell), dímelo y te paso la versión con ; en vez de &&.

06 MANUAL:SESSION-CYCLE

Ciclo de Sesión

¿Cuándo se dispara esto? ¿por qué es distinto del ciclo semanal?

El ciclo semanal que se detalla en MANUAL:WEEKLY-FLOW asume que el sistema documental y el estado del Pipeline están sanos al momento de empezar a trabajar. Esa suposición no es gratuita: cada vez que abres una conversación nueva con Claude para operar VANTAGE, esa conversación pasa primero por su propio ciclo de vida — independiente del ciclo semanal, y que existe precisamente para que nunca operes sobre un supuesto sin verificar.

Piensa en esto como el equivalente a revisar que las luces del tablero no tengan ninguna advertencia antes de arrancar el coche: no es el viaje en sí, es la condición para que el viaje no te sorprenda a medio camino.

Este ciclo se dispara con dos comandos:

- vantage-session-open al inicio de cada sesión
 - vantage-session-close al final
 - Y hace tres cosas que ningún otro punto del sistema hace:
- 1. Deja un registro de que la sesión existió y en qué estado terminó (el Session Ledger).
- 2. Confirma que los 10 documentos fundacionales + el Census están todos en la misma versión (nunca uno adelantado y otro atrasado).
- 3. Te recuerda, sin que tengas que preguntarlo, qué quedó pendiente de la sesión anterior.
- 4. No necesitas invocarlo tú manualmente cada vez que se te ocurra — pero sí necesitas recordar que es el primer paso obligatorio: si acabas de abrir Claude para trabajar en VANTAGE hoy, el primer paso siempre es este ciclo, antes de tocar Tracker, Dashboard o cualquier trigger de CV descrito en MANUAL:WEEKLY-FLOW.

¿Por qué existe esto?

Antes de que este ciclo existiera, cada sesión de Claude arrancaba “en frío”: el agente asumía que el corpus de Notion (Kernel, Manual, System Prompt, Career Canon, Aliases, Changelog) estaba en la versión que recordaba de la sesión anterior.

- No había ningún mecanismo que confirmara si una sesión había terminado bien o si Claude simplemente dejó de responder a medio trabajo — un timeout, un cierre accidental de pestaña, un crash.
 - El resultado era drift silencioso: un documento se actualizaba, otro no, y nadie se enteraba hasta que las contradicciones aparecían en producción (esto es, en parte, lo que motivó el Census — ver KERNEL:CENSUS-

SYNC, y MANUAL:HEALTHCHECK, donde se detalla cómo se regenera el Census).

- Ambos skills viven en /mnt/skills/user/ y son deliberadamente cortos: cada token gastado en abrir/cerrar sesión es un token que no queda para la tarea real — relevante dado el tier de la cuenta.

Open Session Protocol

1. ANNOUNCE: SESSION-OPENING...
 2. LEDGER: SQL directo vía MCP está bloqueado en este plan (query_data_sources no disponible en este workspace). Usar en su lugar:
 - notion-search sobre el data source del Ledger (collection://38324240-c686-47d0-8082-cee5e4409f88)
 - notion-fetch de la fila más reciente devuelta
 - Si Status = OPEN o duplicados -> Reportar WARN. Crear fila nueva (Status: OPEN).
1. HEALTH: Verificar System Prompt + ID Census vía MCP (ya cubierto por el Bootstrap universal, KERNEL:DOCUMENTATION-004).
 2. PENDING: Leer campo Pending Summary de la fila anterior. Si es CLOSED-COMPRIMIDO, priorizar resolución de deuda técnica.
 3. SNAPSHOT: Operador pega dump de -bootstrap. Si vacío -> "SNAPSHOT: 0 TAREAS CRÍTICAS". Terminal ya no es requisito bloqueante para abrir sesión.
 4. READY: SESSION-OPENED: VANTAGE READY (Version/Tier Mode).

Close Session Protocol

1. ANNOUNCE: CLOSING SESSION...
 2. 1.5 . TOKEN-GATE: Si hay alerta de tokens o instrucción "cierre rápido/comprimido", activar MODO COMPRIMIDO (omite pasos 3-4, activa 3'-5'). Default: MODO COMPLETO.
 3. INVENTORY: Operador declara si hubo cambios. Si NO hubo, saltar al paso 6.
 4. CENSUS: Si hubo cambios de ID, requiere output local de generate_census.py. Falla -> Blocked-Census.
 5. CHANGELOG & VERSION:
 - Completo: Draft texto plano -> APROBAR_WRITE.
 - Comprimido: Una línea: [COMPRIMIDO] resumen + "expandir en próxima sesión".
1. VERIFY & SYNC: Gate Absoluto. Requiere output local de verify_versions.py --sync. Validar [VEREDICTO FINAL] PASS.
 - Comprimido: Si no hay output, marcar SYNC PENDIENTE en Ledger.
1. SUMMARY: Bloque homologado (mismas 5 secciones de handoff).
 - Comprimido: Formato bullet de una línea por sección. Priorizar IDs y Pendientes.
1. LEDGER: Update Notion -> Status: CLOSED o CLOSED-COMPRIMIDO,

Closed At [now], Pending Summary [bloque paso 5].

2. TERMINATE: SESSION CLOSED -> nuevo chat.

¿Qué hacer si algo no cuadra?

- Terminal no está disponible → Operación detenida (Fail-Fast). No existe bypass automatizado vía MCP para este chequeo — ni en apertura ni en cierre — con el fin de proteger la cuota de la API de Notion y evitar el desperdicio de tokens de contexto reconstruyendo estado desde bases de datos completas. El operador debe diagnosticar la conectividad o el entorno local antes de proceder con cualquier modificación de documentación.
 - El Ledger anterior quedó OPEN → esto lo verás reflejado directamente en el dump que genera --bootstrap. Revisa manualmente si algo quedó a medio escribir antes de seguir. El sistema te lo señala, pero la decisión de qué hacer con eso es tuya.
 - Drift de versión detectado y no es el documento que ibas a tocar hoy → se reporta, no bloquea. Puedes decidir resolverlo ahora o después.
 - Drift de versión detectado y SÍ es el documento que ibas a tocar → se resuelve el drift primero, antes de aplicar cualquier parche nuevo — de lo contrario terminarías escribiendo sobre una base que ya no coincide con lo que las otras piezas del sistema esperan.
 - Un cambio de código, schema o flujo operativo quedó sin reflejo en la documentación → esto no es parte del drift de versión que acabas de revisar arriba, es el caso que cubre KERNEL:DOCUMENTATION-001: el contrato que detecta contenido operativo nuevo sin ancla en Kernel, Manual, Canon o System Prompt, ya sea porque tú lo pides explícitamente ("documentación transversal", "parche orgánico") o porque el sistema lo señala como recordatorio no-bloqueante a media tarea, sin detener lo que estabas haciendo.
 - Un pendiente detectado durante la sesión necesita convertirse en ticket (o no) → esto lo gobierna KERNEL:GATE-DECISION-009 (3 niveles de escalamiento). En resumen: esfuerzo bajo y sin bloqueo confirmado se queda en pending_summary del Ledger
 - Nivel 1: Esfuerzo alto sin fuente dura de bloqueo se sugiere como ticket y espera tu APROBAR_WRITE
 - Nivel 2: Bloqueo o degradación confirmados por una fuente dura (dump de Terminal, Ledger, Changelog, o tu propia declaración explícita) disparan vantage-create-bug-task de forma automática
 - Nivel 3: Ver KERNEL:GATE-DECISION-009 para el detalle completo y las reglas de re-clasificación entre niveles.
 - Con la sesión abierta y sincronizada, el siguiente paso natural es abrir tu mapa de la semana el Checklist, explicado en MANUAL:CHECKLIST.
 - Ver MANUAL:SKILL-GLOSSARY (§23) para el catálogo completo de skills,

incluyendo la convención de anuncio de cada una.

——07 MANUAL:CHECKLIST

El Checklist

El Checklist es la interfaz operativa de todo el ciclo semanal que se detalla en MANUAL:WEEKLY-FLOW. Es un archivo HTML autocontenido con progreso persistente (localStorage), modo claro/oscuro y navegación por día. Ábrelo una vez al iniciar la semana y consúltalo a lo largo de las actividades de cada día — no es una herramienta puntual de un solo momento, es tu mapa de avance de lunes a viernes.

- Ubicación: archivo local Checklist.html (abre en navegador).
 - Reset: botón “↺ Reset semana” en el header para iniciar un nuevo ciclo.
 - Persistencia: el progreso NO persiste entre sesiones distintas del navegador si se limpia el localStorage.

¿Dónde viven los archivos compartidos?

Dashboard.html, Checklist.html, vantage-tokens.css y vantage-theme.js viven todos en Dashboard/.

- vantage-tokens.css define colores y superficies compartidos por ambos HTML.
 - vantage-theme.js es el toggle de tema compartido, con persistencia y sincronía cross-tab.
 - Esto importa porque el Dashboard (que verás en detalle MANUAL:WEEKLY-FLOW-002) usa exactamente esta misma infraestructura visual — no son dos sistemas de interfaz distintos, son la misma base compartida.

Aspecto

El botón de aspecto (ícono sol/luna, arriba a la derecha en ambos HTML) persiste tu elección y se sincroniza automáticamente si tienes ambos HTML abiertos en pestañas distintas del mismo navegador — cambias el tema en uno, el otro se actualiza sin recargar.

¿Qué no hacer?

- No copies/pegues código de un HTML al otro para “igualar” un color o componente — edita vantage-tokens.css o vantage-theme.js, que ambos ya leen. Editar directo en el HTML reintroduce el mismo drift que se corrigió.
 - Si algo se ve distinto entre los dos HTML, es señal de que alguien editó un color o estilo directo en el <style> inline de uno de los dos archivos en vez de en vantage-tokens.css. Revisa ahí primero.
 - Con el Checklist abierto y el ciclo de sesión ya confirmado (MANUAL:SESSION-CYCLE), estás listo para empezar el Lunes — el primer día del ciclo semanal, detallado a continuación.

——08 MANUAL:WEEKLY-FLOW

Flujo Semanal de Operación

Este es el ciclo completo de trabajo, de lunes a viernes. Asume que ya pasaste por MANUAL:SETUP o MANUAL:COLD-START según corresponda, que la sesión actual de Claude ya pasó por su MANUAL:SESSION-CYCLE, y que tienes el MANUAL:CHECKLIST abierto como guía de avance.

8.1 MANUAL:WEEKLY-FLOW-001

Lunes

El lunes es el ciclo de búsqueda activa completo. Se dispara manualmente y cubre las dos capas de búsqueda humana (L1 y L2), más la revisión de lo que L3 recolectó de forma pasiva durante la semana.

Contrato operativo completo de cada capa (Objetivo, Componentes, Campos inmutables, Estados de error, Métricas) — ver KERNEL:ARCHITECTURE-L1-002 (L1), KERNEL:ARCHITECTURE-L2-002 (L2), KERNEL:ARCHITECTURE-L3-002 (L3). Esta sección cubre únicamente el procedimiento paso a paso.

El ciclo comienza con los prompts de búsqueda, los cuales no se copian de versiones anteriores — se materializan bajo demanda ejecutando el Weekly Prompt Assembler:

vassemble

El script hace fetch de Prompt A (base) y de cada Wrapper (Career Sites, LinkedIn, Aggregators, Gemini, Grok, You.com) y de Prompt E desde la PROMPT LIBRARY, sustituye [YYYY-MM-DD] por la fecha del día en cada componente, y genera 7 archivos .md listos para copiar directo a cada motor: Prompt_[Motor][Fecha].md y Prompt_E_Consolidation[Fecha].md.

#	Sesión	Archivo generado	Motor destino
1	Career Sites	Prompt_Career_Sites_[Fecha].md	Motor con crawler
2	LinkedIn	Prompt_LinkedIn_[Fecha].md	Motor con crawler
3	Aggregators	Prompt_Aggregators_[Fecha].md	Motor con crawler
4	Gemini	Prompt_Gemini_[Fecha].md	Gemini
5	Grok	Prompt_Grok_[Fecha].md	Grok
6	You.com	Prompt_You_com_[Fecha].md	You.com

7	Consolidación	Prompt_E_Conso lidation_[Fecha]. md	Perplexity
---	---------------	---	------------

¿Por qué importa la fecha?

TODAY'S DATE define la ventana de búsqueda activa (14 días preferente, hasta 21 con match fuerte). Un prompt con fecha incorrecta produce resultados fuera de ventana o advertencias innecesarias en todos los ítems.

¿Cómo uso los archivos generados?

Abre cada Prompt_[Motor]_[Fecha].md y pega su contenido en el motor correspondiente (Career Sites/LinkedIn/Aggregators vía Comet Desktop con control de navegador activo; Gemini/Grok/You.com directo en Deep Research). Los Prompts B y C pueden usarse en cualquiera de los tres motores L2. Cada ejecución produce un JSON independiente — compílalos para el paso de consolidación.

¿Como los compilo?

En preparación para entrar al Pipeline es necesario consolidar la información recopilada.

- Regresarás a Perplexity Desktop y, usando como base el Prompt E, pegarás los JSONs de L1 + L2.
 - Perplexity aplicará dedup con clave compuesta brand+title+location siguiendo una jerarquía L1 > L2 (de las vacantes duplicadas persistirán las instancias de L1, tomando de L2 la información que pueda complementar sus propiedades para Class A).
 - Perplexity entregará como respuesta un Plain Array consolidado (JSON plano sin capas anidadas), listo para Python.
 - Guardarás el resultado en:
~/Documents/03 Projects/VANTAGE/Layer_1/Feeds/YYYY-MM-DD_consolidated.json

¿Como corre L3?

- L3 lee los correos no leídos de Gmail que tengan asignado el label .Jobs.
 - Extrae vacantes con Groq y las escribe directamente en el Tracker.
 - Ejecuta manualmente para procesar backlog de Gmail antes del siguiente ciclo automático.

v13

Para este momento, los siguientes filtros ya habrán sido aplicados sin consumir cuota:

- Hard-blocked (L'Oréal · Levi's/Dockers · El Palacio de Hierro — ver lista completa en MANUAL:DATA-MANAGEMENT)
 - Asuntos de agradecimiento
 - Newsletters
 - Confirmaciones de cuenta.

- Límites por ejecución:
- Procesa máximo 10 correos por run (configurable en GROQ_MAX_EMAILS_PER_RUN).
- Si hay backlog, el script reporta cuántos quedan.
- Si L3 falla: verifica que LAYER_3/config/layer_3.env existe y contiene las credenciales de Gmail, Groq y Notion. El venv hereda de LAYER_1/.venv — si Layer 1 funciona, L3 tiene el entorno listo. (Para troubleshooting detallado de L3, ver MANUAL:TROUBLESHOOTING.)
- Abre la Terminal y procesa el JSON consolidado de L1+L2:
v11 feed ~/Documents/03 Projects/VANTAGE/Feeds/YYYY-MM-DD_consolidated.json

¿Qué ocurre aquí?

- Dispara: El script vantage_pipeline.sh actúa como wrapper: activa el entorno virtual (.venv), valida la estructura y dispara feed_processor.py para normalizar campos, aplicar dedup cross-layer (ventana 30 días — ver MANUAL:DATA-MANAGEMENT) y presentarte el DRY RUN antes de escribir en Notion.
 - Aprobar escritura: revisa el DRY RUN en terminal. El output muestra las propiedades Class A de cada instancia a crear. Las entradas duplicadas aparecen como SKIP. Las que requieren revisión aparecen como REVIEW_NEEDED. Confirma con y (yes) para escribir en Notion. Cualquier otra tecla cancela sin escribir.
 - Los registros con status REVIEW_NEEDED que se escriben en Notion se resuelven al día siguiente en el Dashboard MANUAL:WEEKLY-FLOW-002
 - Procesar con Python: Para este punto las propiedades Class A de cada instancia nueva se habrán poblado por L1, L2 o L3.
 - Para poblar las propiedades Class B de todas las instancias pendientes en el Tracker, ejecutarás la app LAYER 1.app desde /Applications o usando Terminal:

v11

- READY-TO-APPLY: abre la vista Ready-to-Apply en Notion.
 - Vacantes con Score ≥ 60 están listas para CV Optimization en preparación para tu postulación — esto es lo que trabajarás el Miércoles MANUAL:WEEKLY-FLOW-003.

¿Qué es L4?

L4 mantiene dos cosas sincronizadas en background, sin intervención manual en el ciclo semanal normal:

- El repositorio git del sistema (vgit)
 - Los 6 documentos fundacionales entre Notion y el disco local (vdoc).
 - Son dos herramientas separadas que se combinan: vdoc mueve

contenido documental Notion ↔ ACTIVE/, y al terminar dispara automáticamente un git_sync — por eso casi nunca necesitas correr vgit a mano después de un vdoc.

¿Qué es vgit?

- Ejecuta vgit desde Terminal en cualquier momento para enviar un sync inmediato — útil si hiciste cambios locales fuera del ciclo automático y no quieres esperar al siguiente horario.
 - Verificar último run: `cat /tmp/vantage_l4_gitsync.log` — cada corrida (automática o manual) queda registrada ahí con timestamp, exit code y el output completo del sync, así puedes auditar qué pasó sin depender de las notificaciones del sistema.
 - Si el repo no existe o está corrupto, vgit ya no lo confunde con "sin cambios" — reporta el error explícitamente y la notificación del wrapper se ve roja (✗), no verde.
 - Archivos:
 - Layer_4/scripts/git_sync.py
 - Layer_4/wrappers/git_sync_wrapper.sh
 - ~/Library/LaunchAgents/com.vantage.gitsync.plist
 - Extensión reciente — Skills Distribution: además de vgit, el alias vtriggers (`update_triggers_json.py`) mantiene `skills/triggers.json` — el manifiesto SSOT que cruza cada skill contra su trigger, path y última modificación — y, al finalizar, ejecuta su propio commit+push automático sobre ese archivo (ver KERNEL:ARCHITECTURE-L4 para el detalle de escaneo y validación).
 - Esto es lo que permite que Claude lea siempre la misma versión del manifiesto (vía `web_fetch`) y del contenido de cada skill (vía `git clone --depth 1 local`), sin paso de sincronización manual ni intermediario MCP.

¿Qué es vdoc?

- Sincroniza los 6 documentos fundacionales (Kernel · System Prompt · Career Canon · Manual · Aliases · Change Log) entre Notion y ACTIVE/ en disco.
 - Al terminar encadena un git_sync automático para que el commit quede reflejado en GitHub sin un paso adicional.
 - Tres direcciones posibles:
 - vdoc auto — compara la fecha de modificación de cada documento (local vs. Notion) y sincroniza en el sentido que corresponda, documento por documento. Es el modo por defecto y el más seguro para uso diario: nunca sobrescribe algo más reciente con algo más viejo.
 - vdoc notion — fuerza Notion → local para los 6 documentos, sin comparar fechas. Úsalo solo si sabes que Notion tiene la versión correcta y quieres descartar cualquier cambio local.
 - vdoc local — fuerza local → Notion para los 6 documentos, sin comparar

fechas. Úsalo solo si editaste los .md directamente en disco (offline) y quieres que Notion adopte esa versión.

- Como notion y local sobrescriben sin comparar fechas, ambos son operaciones forzadas: antes de ejecutar nada, vdoc te muestra automáticamente un preview (equivalente a --dry-run) de lo que va a hacer, y te pide confirmación explícita en terminal (s para continuar, cualquier otra tecla cancela).
- Si por alguna razón corres el comando sin una terminal interactiva disponible, el script no asume que confirmaste — cancela por seguridad y no escribe nada. vdoc auto nunca pide esta confirmación porque nunca sobrescribe algo más reciente.
- Modificador dry — se combina con cualquiera de los tres comandos anteriores y con cualquier documento específico, en cualquier orden, y siempre gana: nunca escribe en Notion, en disco ni hace commit, sin importar qué más hayas escrito en la misma línea.
- vdoc dry — preview de auto (equivalente a vdoc auto dry)
- vdoc notion dry — preview de lo que haría vdoc notion, sin ejecutar la escritura forzada
- vdoc local dry — preview de lo que haría vdoc local
- vdoc kernel dry — preview de solo Kernel en modo auto
- Recomendación operativa: corre siempre la variante dry primero cuando no estés seguro de qué dirección va a ganar — te cuesta segundos y evita sorpresas, especialmente antes de un notion o local forzado.

¿Que es sync?

- Sync quirúrgico por documento — cualquiera de los 6 nombres puede pasarse solo o combinado con dirección/dry:
 - vdoc kernel
 - vdoc system_prompt
 - vdoc career_canon
 - vdoc manual
 - vdoc aliases
 - vdoc change_log
- Sin dirección explícita, cada uno corre en modo auto (gana el más reciente) solo para ese documento — los otros 5 no se tocan.
- Se puede combinar con notion/local (ej. vdoc notion kernel fuerza solo Kernel Notion→local) y con dry (ej. vdoc kernel dry).

——8.2 MANUAL:WEEKLY-FLOW-002

Martes

¿Qué resuelvo aquí?

Antes de avanzar al miércoles, este es el momento de resolver lo que quedó bloqueado el lunes: REVIEW_NEEDED · BLOCKED recuperables · NADs vencidas. Las vacantes que recuperes aquí son las que estarán disponibles en Ready-to-

Apply para trabajar mañana.

Si el bloqueo es por un campo Class A corregible, usa el Dashboard: Proponer Patch → Validar → Aceptar. No uses el Dashboard para forzar un CREATE en vacantes que no cumplen score — úsalo solo para corregir datos erróneos. (Recuerda de MANUAL:HOW-IT-WORKS: un Gate BLOCKED no es un error del sistema a "saltarse", es una vacante cuyos datos de entrada tienen un problema identificable y corregible.)

Partes del Dashboard

Es una sola herramienta (dashboard.html + dashboard_server.py :8000), no hay pestañas ni vistas separadas. La pantalla es un panel de recuperación de vacantes bloqueadas, con una tira de estado del pipeline (L1 → RT-1 → Notion → Mail) como indicador visual — no una vista de navegación distinta. Comparte la infraestructura visual (vantage-tokens.css, vantage-theme.js) con el Checklist, descrita en MANUAL:CHECKLIST.

Archivos: Dashboard/dashboard.html · Dashboard/scripts/dashboard_server.py.

Abrir el Dashboard: ejecuta en terminal:

vd

El wrapper dashboard_start.sh arranca el servidor Flask en <http://127.0.0.1:8000> (accesible también vía Tailscale desde otros dispositivos), ejecuta un smoke test automático y abre dashboard.html en el navegador. Output esperado en terminal: SMOKE PASSED — abriendo dashboard. Si el smoke falla, emite notificación sonora de error (Basso) y no abre la UI. El indicador "BACKEND OK/OFFLINE" en la esquina superior confirma la conexión en vivo.

Partes del Dashboard:

- Sidebar (columna izquierda): estado de la instancia activa — instance_id, payload actual de la vacante (campos Class A como aparecen en Notion), capabilities disponibles en el estado actual (can_patch · can_validate · can_accept · can_archive) y Audit Log en tiempo real con cada evento registrado.
 - Panel principal (área derecha): cuatro secciones — selector de vacante (dropdown con todas las vacantes en Gate = BLOCKED, botón Crear instancia), máquina de estados FSM (visualiza el estado actual: BLOCKED → PATCHED → RETURNED_TO_CREATE), panel de patch (formulario con campos Class A editables: URL · JD · Source_Type · Prioridad), y área de resultado de validación (PASS verde o FAIL rojo con motivo).
 - Botones: Crear instancia · Proponer Patch · Validar · Aceptar Patch · Archivar · Sincronizar.

BLOCKED

1. Selecciona la vacante del dropdown (muestra Marca · Rol · Score · VM_Scope).
2. Crear instancia — abre una instancia en estado BLOCKED y carga el payload desde Notion. Audit Log registra domain.instance.created.

3. Edita los campos incorrectos en el panel de patch — solo Class A (URL · JD · Source_Type · Prioridad). Los campos Class B no son editables.
4. Proponer Patch — almacena la corrección. Audit Log registra domain.patch.proposed.
5. Validar — el backend ejecuta run_pipeline.py con el patch y verifica si el resultado sería CREATE. Si pasa: estado → PATCHED, resultado verde. Si falla: estado permanece BLOCKED, resultado rojo con motivo.
6. Aceptar Patch — escribe los campos Class A corregidos en Notion. Estado → RETURNED_TO_CREATE. Audit Log registra domain.patch.accepted.
7. Corre el pipeline para que Python recalcule:
~/vantage_pipeline.sh

FAILED

Secuencia — vacante no recuperable: usa el botón Archivar. El Dashboard escribe Next_Action = Archivar en Notion y cierra la instancia en estado FAILED. No pasa por el pipeline.

REVIEW_NEEDED

Las entradas con este status son escritas en Notion por feed_processor.py cuando no pudieron procesarse completamente: la URL era parcial o ambigua, la marca no resolvía contra el alias map, o el sistema detectó un semi-duplicate cross-layer que requiere revisión humana. Mientras el status permanezca en REVIEW_NEEDED, sus campos Class B (Score, Gate_Decision, VM_Scope, Role_Class) quedan bloqueados — Python no los calcula.

Contrato de resolución — 4 pasos obligatorios:

1. Abre la entrada en Notion e identifica el problema indicado en el campo Notas (ej. "URL parcial", "alias no resuelto: Nike México", "semi-duplicate").
 2. Corrige el campo problemático directamente en Notion: reemplaza la URL parcial con la URL completa, o ajusta el nombre de la marca al valor que exista en el alias map.
 3. Cambia Status → Target. Este es el único valor que Python reconoce como señal de resolución. Cualquier otro valor (incluyendo dejar REVIEW_NEEDED) mantiene el bloqueo en el siguiente run.
 4. Corre el pipeline:
~/vantage_pipeline.sh

Python detecta Status = Target en entradas que tenían Gate vacío o REVIEW_NEEDED y procesa sus campos Class B normalmente — calcula Score, Gate_Decision y el resto.

Por qué Target y no otro valor: Target es el estado operativo estándar de una vacante en espera de procesamiento. Usarlo como señal de resolución mantiene el contrato de estados consistente — no requiere un valor nuevo ni lógica adicional

en Python.

Nota importante: estas entradas no pasan por el Dashboard. El Dashboard es para vacantes con Gate = BLOCKED que ya tienen campos Class B calculados y necesitan corrección de inputs Class A. REVIEW_NEEDED es un estado previo — todavía no llegó a tener Gate calculado.

8.3 MANUAL:WEEKLY-FLOW-003

Miércoles

Optimización de CV para vacantes priorizadas en Ready-to-Apply. Claude opera activamente en este ciclo — es el único día donde el AI Component tiene rol principal. L3 sigue corriendo en sus horarios habituales, en background.

CV-A

Abre Ready-to-Apply en Notion y elige la vacante a trabajar. Copia la URL del campo URL (career page oficial) o el texto del JD. Abre una nueva sesión de Claude (recuerda: esto significa pasar primero por el Ciclo de Sesión de MANUAL:SESSION-CYCLE si aún no lo has hecho hoy) y dispara:

CV-A [URL de la vacante]

o pega el texto del JD directamente. Claude no accede al Tracker de forma autónoma — el trigger debe ser explícito.

CV-A es análisis: qué keywords posicionar, qué gaps cubrir, qué tono de marca adoptar. CV-B es producción: el documento final. En una sesión única, el contexto de análisis contamina la voz del CV. La separación es una restricción de calidad, no de conveniencia.

Ruta alterna (batch, opcional): si tienes varias vacantes Ready-to-Apply el mismo miércoles, `adapt_tracker_export.py` → `cv_a_batch_agent.py` → `cv_a_prep.py` (ver MANUAL:SCRIPT-GLOSSARY-CV-PREP) genera un `HANDOFF_scaffold_<ID>.md` por vacante en un solo paso de Terminal, sin copiar URL/JD una por una. Cada scaffold sigue requiriendo su propia sesión CV-A [scaffold] — la preparación es mecánica, el análisis sigue siendo de Claude, uno a la vez (ver KERNEL:CV-PIPELINE-003).

Claude realiza tres tareas de análisis, en este orden:

- Extrae los 6 keywords de posicionamiento del JD.
 - Identifica los gaps entre los requisitos del rol y el perfil de experiencia canónico del Career Canon.
 - Determina el Positioning Mode aplicable mediante el Algoritmo de Selección N1–N4 (KERNEL:CV-PIPELINE-001) — hay cuatro modos posibles, definidos en el Career Canon:
 - N1 Luxury Brand Execution
 - N2 Store Design & Flagship
 - N3 Regional Brand Execution
 - N4 Commercial VM & Field Leadership
 - Además, define el tono de marca del CV y detecta el idioma del JD (ES/EN) para el output.

- Output de la sesión — el HANDOFF, 7 campos obligatorios:

```
{
  "empresa": "",
  "rol": "",
  "JD_keywords_top6": ["", "", "", "", "", ""],
  "fit_gaps": ["", ""],
  "tono_marca": "",
  "idioma": "",
  "positioning_rationale": ""
}
```

La sesión termina aquí. No se escribe ningún CV en CV-A.

HANDOFF

Contrato de Transferencia: CV-B no inicia con un HANDOFF incompleto. Si cualquier campo está ausente, el sistema lo solicita antes de continuar.

CV-B

Abre una sesión nueva de Claude. Pega el HANDOFF completo y dispara:
CV-B [pega el HANDOFF]

Claude ejecuta, en secuencia:

- Verifica los 7 campos del HANDOFF.
 - Cruza el HANDOFF contra el contrato de output del Career Canon para validar que bullets y KPIs sean derivados canónicos (no inventados).
 - Aplica el Positioning Mode definido en CV-A.
 - Usa el campo idioma del HANDOFF para seleccionar la versión ES o EN de cada sección del Career Canon (no se generan CVs bilingües ni se mezclan idiomas dentro de un mismo output).
 - Genera el CV bajo ese mismo contrato de output.
 - El output tiene tres partes obligatorias y secuenciales:
- 1. Markdown con Figma tags — Claude entrega el archivo .md completo en la misma sesión. Cada slot va encabezado por su tag (##### figma_text_id). El operador lo revisa y autoriza antes de cualquier escritura en Notion.
 - 2. Autorización explícita del operador — Claude espera confirmación antes de continuar. Sin autorización, no escribe nada.
 - 3. Documentar la URL del Markdown.
 - 4. Regla de orden: el Markdown nunca se escribe en Notion si el operador no ha autorizado explícitamente. El orden cronológico de experiencia es invariante: C01 → C02 → C03 → C04 → C05. No se reordena por vacante ni por Positioning Mode.
 - 5. Escritura en Notion (dos destinos):
- Página en DERIVED OUTPUTS · ARCHIVE del Career Canon — con footer de Positioning Mode activo y fecha.

- Bloque # MARKDOWN CANON ALIGNED en la página de la vacante en el Tracker — el Markdown completo con Figma tags, dentro de un bloque de código markdown.

Figma

CV-B ya no tiene permiso creativo sobre la estructura. El proceso es de inyección en slots vía plugin — arquitectura, contrato de bloque, flujo de inyección, sanitización y diagnóstico de errores viven en MANUAL:FIGMA-SYNC-003 (§20, Flujo de Inyección) y subsecciones adyacentes 20.1–20.5. Con el .md autorizado en mano:

1. Abre Figma Desktop y el archivo del CV.
2. Plugins → Development → VANTAGE CV Sync.
3. Pega el .md de CV-B y haz clic en Inyectar a Nodos Nativos.
4. Verifica la notificación y revisa el lienzo antes de exportar (PDF).
5. Si el plugin reporta "Keys sin resolver" o "0 nodos actualizados", ver MANUAL:FIGMA-SYNC-DIAGNOSTIC (§12).
6. Riesgo de Batch-Cloning (v9.16.0): al inyectar un CV-B proveniente de un batch, si el lienzo muestra viñetas dobles ("• •") o el operador reconoce bullets idénticos a los de otra vacante del mismo Positioning Mode, rechazar el archivo y solicitar su regeneración antes de continuar con la inyección.

QA

QA [adjunta el PDF exportado]

Claude revisa formato y completitud con checklist de 6 ítems y entrega go/no-go.

QA no evalúa fit — evalúa que el documento esté correcto como entregable.

Si QA aprueba, cambia Status a Postulado en Notion y corre:

```
~/vantage_pipeline.sh
```

Python detecta el Status y asigna Gate_Decision = APPLIED. La vacante sale de Ready-to-Apply automáticamente.

08.4 MANUAL:WEEKLY-FLOW-004

Jueves

Ejecuta solo si hay nuevas vacantes que procesar — 10 minutos máximo:

```
~/vantage_pipeline.sh
```

Script: ~/vantage_pipeline.sh. Este día no tiene un procedimiento distinto al ya descrito en el Lunes (MANUAL:WEEKLY-FLOW-001) — es simplemente una repetición ligera del paso de procesamiento, para no dejar acumular vacantes hasta la siguiente semana si el Lunes no alcanzó a cubrir todo el backlog.

08.5 MANUAL:WEEKLY-FLOW-005

Viernes

```
~/vantage_pipeline.sh analytics
```

Output: efectividad por fuente, tasa de links muertos por tipo de URL, ratio career pages vs. aggregators.

Acción concreta: si career pages producen menos de 5 resultados relevantes en la semana, ajusta el Prompt A (ver MANUAL:PROMPTS-WRAPPERS) — no el threshold de Score. (Recuerda MANUAL:FAILURE-PHILOSOPHY: el Score bajo no es el problema a corregir, el input de búsqueda sí lo es.)

Con esto se cierra el ciclo semanal. La siguiente vez que abras Claude para trabajar en VANTAGE, el ciclo completo empieza de nuevo desde MANUAL:SESSION-CYCLE.

08.6 MANUAL:WEEKLY-FLOW-006

Matriz de Cadencia Operativa

Complementa la narrativa día-por-día con una vista de:

- Trigger → resultado desacoplada del calendario.
 - Las entradas del Tracker reflejadas en la columna “Resultado” son siempre campos Class B
 - calculados por Python — no escritura manual (ver KERNEL:SCHEMA-002).
 - | Trigger | Contexto de Invocación | Objetivo | Resultado en Tracker | Referencia |
 - | --- | --- | --- | --- | --- |
 - | feed_processor.py • APROBAR_WRITE | Lunes (o cualquier día con nuevas vacantes) — operador revisa output de L1/L2/L3 | Ingresar vacantes nuevas al SSOT con Class A poblado | Filas nuevas con Status=RAW → pipeline calcula Class B inmediatamente | KERNEL:TRIGGER-001, KERNEL:SCHEMA-002 |
 - | ~/vantage_pipeline.sh (run completo) | Después de cada APROBAR_WRITE de feed | Calcular Score, Gate_Decision, Next_Action sobre entradas Class A | Gate_Decision=CREATE/BLOCKED, Score, VM_Scope, Role_Class actualizados | KERNEL:GATE-DECISION-010 |
 - | vd (:8000) — Dashboard RT-1 | Condicional: solo si existen vacantes con Gate=BLOCKED recuperables | Corregir Class A de vacantes bloqueadas y re-ingresar al pipeline | PATCHED → re-evaluación → CREATE o BLOCKED renovado | KERNEL:GATE-DECISION-005, KERNEL:GATE-DECISION-011 |
 - | CV-A [URL/JD] | Miércoles (o cuando vacante alcanza READY_TO_APPLY) | Generar artefacto CV-A adaptado a la vacante | Artefacto en Figma Sync; sin cambio en Tracker | KERNEL:CV-GOLDEN-RULES |
 - | STATUS [SYSTEM] | Cualquier momento — auditoría ad-hoc | Verificar estado global del sistema y detectar huérfanos/inconsistencias | Sin escritura — reporte en sesión | SP:TRIGGERS |

- Nota operativa: los días de semana en 8.1–8.5 son metadato de cadencia del operador, no guard conditions de ningún gate. Una vacante con Score ≥ 60 puede alcanzar READY_TO_APPLY el mismo día de su ingesta, sin esperar al ciclo siguiente. → Ver KERNEL:GATE-DECISION-011 para vista completa de transiciones.

——09 MANUAL:RUNTIME

Runtime

Ya viste varios de estos comandos en acción durante el flujo semanal (MANUAL:WEEKLY-FLOW) — esta sección los reúne como catálogo de referencia completo, junto con el detalle de cuándo y por qué correr cada uno.

9.1 MANUAL:RUNTIME-001

¿Qué es el Runtime?

Es la herramienta de observabilidad del sistema. Permite interrogar a Notion y extraer contexto semántico sin salir de la terminal.

¿Por qué vversions y vcensus viven aquí?

Version Check Tool (vversions) y Census (vcensus) —ya documentados como comandos en MANUAL:RUNTIME (9.2) y en uso durante el Ciclo de Sesión (MANUAL:SESSION-CYCLE)— pertenecen a esta misma capa: interrogan a Notion para darte visibilidad (versión documental, salud del Census), nunca escriben datos de negocio del pipeline de vacantes. Si alguna vez te preguntas por qué vversions vive junto a vantage.py status en vez de junto a vl1, es por esto: los dos son observación, no procesamiento de vacantes.

9.2 MANUAL:RUNTIME-002

Comandos Principales

Estos comandos operan sobre el estado del Tracker y están disponibles como subcomandos de vl1. Cada uno tiene un alcance preciso y un modo de operación por defecto.

- vl1 tracker — genera un reporte de estado del Tracker en tiempo real: distribución por Gate_Decision, conteo de entradas activas (CREATE + APPLIED), entradas BLOCKED, aplicaciones de los últimos 7 días y NADs vencidas. Es el punto de partida del ciclo semanal — corre antes de cualquier otra operación para tener visibilidad del estado actual (esto es lo que produce el output que viste en el Test Inicial de Setup, MANUAL:SETUP, Paso 5).
 - vl1 analytics — analiza la efectividad de las fuentes de discovery: qué canales producen más entradas CREATE, qué ratio de URLs funcionales tienen, cuál es el score promedio por fuente, y qué método de búsqueda (SEARCH-WEEK, SEARCH-EXEC, Manual) tiene mayor tasa de éxito. Corre los viernes como parte del cierre semanal (MANUAL:WEEKLY-FLOW-005).
 - vl1 batch — modo de operación por defecto: read-only. Muestra la distribución por Status y el conteo de entradas que serían afectadas por la operación batch configurada en el script. Para ejecutar escritura, pasar

el flag `--execute` explícitamente:

```
vl1 batch --execute
```

Sin `--execute`, el comando nunca escribe en Notion. Esta protección es permanente — no se puede desactivar sin modificar el flag.

- `vl1 recovery` — verifica la consistencia de los datos en el Tracker: detecta entradas sin `Score`, sin `VM_Scope` o sin `Gate_Decision`. También gestiona checkpoints del pipeline — si un run anterior falló a mitad, `recovery` carga el último checkpoint y permite retomar desde el paso fallado. Corre cuando el pipeline reporta inconsistencias o tras un fallo inesperado.
 - `vl1 profile` — gestiona la configuración del perfil activo del sistema: keywords VM y de pivote, pesos de scoring, empresas target por tier y foco geográfico. Permite actualizar el perfil sin editar código — los cambios se persisten en `config/profile_config.yaml`. Opción 7 ("Salir sin cambios") es el exit seguro; cualquier cambio guardado requiere propagación manual a `layer_1_run.py`.
 - `vl1 backfill` — catch-up de campos Class A faltantes en entradas existentes: `layer`, `hash` y `Prioridad`, para registros que no pasaron por la escritura primaria de Fase 3.6. Opera con preview obligatorio antes de escribir — muestra exactamente qué entradas serán modificadas y por qué razón se infirió el `layer`. La fórmula de `Prioridad` (`Urgencia × Importancia`) vive en `priority_logic.py`, referenciada desde `KERNEL:TRIGGER-002` — este comando la ejecuta como catch-up; `vl1` (bare) la ejecuta primero como parte del ingreso normal del pipeline (Fase 3.6). Acepta `--dry-run` para preview sin confirmación:

```
vl1 backfill --dry-run
```

Sin `--dry-run`, solicita confirmación explícita (s) antes de cualquier escritura.

- `vversions` — acepta `--bootstrap`, `--sync`, `--scripts`, `--skills`, `--length` (Sanity check de integridad estructural: conteo de bloques de texto extraíble vs. baseline. Read-only. Exit code 1 si ATENCIÓN REQUERIDA), `--update-baseline` (Actualiza `length_baseline.json`. Requiere `--length` y confirmación explícita del operador cuando el veredicto no es PASS). — alias corto de `verify_versions.py`, el motor de verificación y sincronización de versión de los 9 documentos fundacionales (`KERNEL:VERSION-CHECK-TOOL`). No es un comando del Tracker de vacantes como los `vl1 *` de arriba — es infraestructura documental, y su uso está integrado al Ciclo de Sesión completo en `MANUAL:SESSION-CYCLE`, no como comando suelto.
 - Acepta cuatro flags:

- --bootstrap (dump read-only de apertura)
- --sync (único modo de escritura y verificación real, relee cada documento post-escritura)
- --scripts (gap report read-only de scripts .py/.sh contra SCRIPT LIBRARY)
- --skills (gap report read-only de archivos .skill contra SKILL LIBRARY)
- Estos dos últimos alimentan a vantage-sync-script-library y vantage-sync-skill-library respectivamente. El modo --check fue eliminado en Kernel v9.6.2.
- vcensus — alias corto de generate_census.py. Regenera el V-ID-CENSUS y reporta IDs huérfanos detectados en los documentos fuente. Se corre en el paso 1 del Cierre de Sesión (MANUAL:SESSION-CYCLE) si algún ID cambió de estado durante la sesión — ver también MANUAL:HEALTH-CHECK, "¿Qué es el Census ID?", para el detalle completo de cuándo es obligatorio. Acepta --auto-fix-orphans (corrige huérfanos de forma interactiva, dándolos de alta en CENSUS_SPEC sin editar el script a mano) y --sync-to-notion [page_id] (sincroniza el export del Census directo a la página de Notion indicada, sin paso manual de copiar/pegar).
- vsum — alias corto de vsum.py. Resume transcripts de sesiones de trabajo (propias o de otra IA) a Markdown estructurado, orientado a continuidad entre chats sin pérdida de contexto. No es comando del Tracker de vacantes ni observabilidad de Notion como vversions/vcensus — es infraestructura de continuidad documental sobre transcripts externos.
- Acepta archivo .md, URL de Claude share. Acepta los siguientes flags:
 - --batch para varios a la vez
 - --notion crea la página de resumen como hija del INBOX en Notion.
 - Uso típico: vsum chat.md --notion.

9.3 MANUAL:RUNTIME-003

Cuándo Correr Sync

Correr vantage.py sync después de:

- Cualquier ciclo L1/L2 que haya escrito entradas nuevas en Notion.
 - Después de resolver entradas REVIEW_NEEDED en el Tracker (ver MANUAL:WEEKLY-FLOW-002).
 - Si status muestra "warning": "entity_index_stale" (index > 24h).
 - Si status muestra orphan_candidates > 0 de forma persistente.
 - No es necesario para cambios de Status, Score, Gate_Decision en páginas individuales — esos se leen en vivo vía resolve/context.

9.4 MANUAL:RUNTIME-004

Runtime Build

El Runtime Build regenera los tres artefactos de lectura del sistema:

entity_index_v2.json, graph_v2.json y backlinks_v2.json. Se corre desde Layer_1/

scripts/ con el venv activo.

Cuándo correrlo:

- Después de cualquier migración de namespaces o cambio en resolver_registry_v2.json.
 - Si graph_v2.json muestra self-loops inesperados (síntoma de colisión de namespace).
 - Si entity_index_v2.json contiene IDs con prefix incorrecto.
 - Como parte del cierre formal de un release que afecte la capa de Runtime.
 - El Build es determinista: el mismo Registry + el mismo estado de Notion producen los mismos artefactos. Si el resultado varía entre runs sin cambios en los inputs, es una señal de problema en el Registry — no en el Build.

Sobre resolver_registry_v2.json

Desde v2.4.0 (Runtime Contract Migration), este archivo es la fuente enforced — no solo declarada — de namespace ownership. Cada tipo de entidad tiene su entity_prefix definido aquí; ningún componente del sistema puede hardcodear ni inferir un prefix.

- Riesgo: una edición manual que asigne un prefix incorrecto producirá colisiones de namespace en el siguiente Runtime Build, lo que se manifestará como self-loops en graph_v2.json.
 - Antes de editar: verificar el prefix activo por tipo de entidad y correr Runtime Build para confirmar que no hay colisiones.
 - Comandos relacionados de deduplicación y oportunidades:

```
cd $LAYER_1_DIR && source .venv/bin/activate && python3
scripts/consolidate_duplicates.py # alias: vdedup
cd $LAYER_1_DIR && source .venv/bin/activate && python3
scripts/dedup_opportunities.py    # alias: vopport
```

——9.5 MANUAL:RUNTIME-005

Notebook Gemini — Triage de Consultas Documentales

Antes de que Claude ejecute un plan que dependa de un dato duro del corpus fundacional (ancla exacta, discrepancia entre documentos, versión vigente de un nodo), puede resultar más eficiente delegar la verificación a Notebook Gemini en vez de que Claude haga fetch estructural — mismo principio de economía de tokens que ya aplica a vversions/vcensus (9.1). No es un alias de Terminal: la consulta se hace manualmente, fuera del pipeline de scripts.

Qué resuelve: dudas de gobernanza documental con ancla verificable (PREFIX:KEY), detección de drift entre documentos.

Qué no resuelve: evaluación de fit, cálculo de Score/Class B,

redacción de CV, creación de reglas de negocio — fuera de su contrato de solo-lectura (ver KERNEL:DOCUMENTATION-012).

10 MANUAL:DATA-MANAGEMENTMatriz de Guard de Mutación (PR #10):

Status
Ver tabla completa en KERNEL:GATE-DECISION-012.
Gestión de Datos
Esta sección consolida en un solo lugar todo lo relacionado con exclusiones y deduplicación de vacantes — conceptos que se mencionan a lo largo de MANUAL:OBJECTIVE, MANUAL:HOW-IT-WORKS y MANUAL:WEEKLY-FLOW, y que aquí tienen su definición completa y única.

10.1 MANUAL:DATA-MANAGEMENT-001

Notas de Archivado

Cuando VL1 ejecuta una decisión de archivado, no solo la marca — también documenta por qué, en el momento exacto de la decisión (ver KERNEL:GATE-DECISION-013).

Escenarios: URL Gate bloqueado, Misfit de perfil, NAD vencido.

Cómo se escribe: nunca sobrescribe Notas — agrega separado por línea vacía.

Soporta modo dry-run.

Qué no hace: no gobierna quién marca el candidato — eso es vantage-tidy-opportunities-tracker/vantage-housekeeping-archive (MANUAL:SKILL-GLOSSARY-HOUSEKEEPING), que operan sobre el registro ya documentado.

Hard Blocks

Estas empresas o tipos de rol nunca entrarán al sistema. Se filtran en el origen (antes de que la vacante exista como registro en Notion) y no son recuperables bajo ninguna circunstancia, ni siquiera vía Dashboard:

- L'Oréal (todas las divisiones)
 - Levi Strauss & Co. (Levi's, Dockers)
 - El Palacio de Hierro
 - Roles store-level sin gestión estratégica

Soft Blocks

A diferencia de los Hard Blocks, estas vacantes sí pueden recuperarse: fueron bloqueadas por inconsistencias en datos Class A (URL rota, JD parcial) o por score insuficiente, no por pertenecer a una empresa vetada. Se recuperan corrigiendo el input incorrecto a través del Dashboard — el procedimiento completo está en MANUAL:WEEKLY-FLOW-002 (Martes).

Dedup

El sistema opera dos mecanismos complementarios de deduplicación con propósitos y ventanas distintas:

- Dedup en tiempo real (ingesta): ventana de 30 días, hash exacto + URL + brand+title. Previene contaminación del Tracker con duplicados obvios al momento de ingesta (feed_processor.py).

- Dedup por auditoría post-ingesta: ventana configurable de 60 días por default, matching fuzzy (brand≥0.85, rol≥0.7) + fingerprint de contenido, contraste contra el Archivo Tracker y reglas anti-falsos positivos extensibles (ANTI_FALSE_POSITIVE_RULES). Desde v9.21.0 corre automáticamente al finalizar layer_1_run.py mediante ENABLE_DEDUP_AUDIT=true, hereda --dry-run del pipeline principal y exporta métricas a dedup_metrics.json.
- Jerarquía entre capas: L1 > L2 > L3. Cuando dos capas detectan la misma vacante, persiste la instancia de la capa de mayor jerarquía, pero se toman de la capa de menor jerarquía los datos que puedan complementar sus propiedades Class A (esto es exactamente lo que ocurre en el paso de Consolidation & Dedup del Lunes, MANUAL:WEEKLY-FLOW-001).
- Resolución de flags: los registros marcados Dedup_Flag='Posible duplicado' por la auditoría post-ingesta son candidatos a archivado; su resolución opera vía vantage-tidy-opportunities-tracker (DRY RUN + APROBAR_WRITE), no hay archivado automático.
- Ventana: 30 días. Una vacante que ya existe en el Tracker no se vuelve a crear si aparece de nuevo dentro de esta ventana.
- Clave compuesta: brand + title + location.
- Jerarquía entre capas: L1 > L2 > L3. Cuando dos capas detectan la misma vacante, persiste la instancia de la capa de mayor jerarquía, pero se toman de la capa de menor jerarquía los datos que puedan complementar sus propiedades Class A (esto es exactamente lo que ocurre en el paso de Consolidation & Dedup del Lunes, MANUAL:WEEKLY-FLOW-001).

——11 MANUAL:MONITOR

Health Check

Red Flags

- Ready-to-Apply vacío por más de 3 días → ajustar Prompt A (ver MANUAL:PROMPTS-WRAPPERS), no el threshold. (Ver también MANUAL:FAILURE-PHILOSOPHY.)
 - Career pages con éxito < 50% → revisar fuentes de discovery.
 - Pipeline runtime > 5 min → archivar entradas inactivas.

¿Qué es y qué lee?

Es un script de arranque, de lectura estricta (cero escritura salvo la excepción documentada abajo). Corre automáticamente al invocar el alias start (activa venv + carga env + ejecuta el script). También puede correrse manualmente:
 cd Layer_1/scripts && python3 health_check.py

Qué lee, en este orden:

1. Versión del sistema — propiedad Versión de V-CHANGELOG vía Notion.
2. Entorno (.env) — verifica que NOTION_TOKEN y demás vars requeridas

existan.

3. Git — `git status --porcelain`; reporta si hay archivos sin commitear.
4. Último commit (vgit) — `git log -1` para timestamp de referencia.
5. Notion reachable — fetch mínimo a V-SYSTEM-PROMPT para confirmar conectividad y token válido.
6. Docs fundacionales — confirma que los 7 documentos existen localmente en ACTIVE/.
7. Último vdoc sync — cuál de los 6 docs locales tiene el mtime más reciente, y hace cuánto.
8. Antigüedad de índices (index_age) — ver detalle abajo. Única sección con capacidad de escritura (auto-sync condicional).
9. Tickets pendientes — Bug Tracker y Task Tracker, agrupados por prioridad.
10. Índices monitoreados: `graph_v2.json` y `entity_index_v2.json`, ambos en `Layer_1/scripts/`.
11. Comportamiento del auto-sync (desde v8.7.9): si algún índice supera 24 horas sin actualizarse, `health_check.py` dispara automáticamente `python3 vantage.py sync` (housekeeping de rutina — no requiere aprobación del operador, no es remediación de un fallo, según la misma lógica de MANUAL:FAILURE-PHILOSOPHY: esto no es un error, es mantenimiento normal). El sync se dispara una sola vez por corrida, solo si al menos un índice cruzó el umbral — no re-sincroniza índices ya frescos, y no corre si todos están dentro del umbral.

¿Cómo Leer el Output?

Cómo leer el output:

- ✓ verde — check pasó.
 - ! amarillo — advertencia, no bloquea (ej. índice stale antes del auto-sync, tickets pendientes).
 - X rojo — fallo real, contribuye al exit code final.
 - Línea final Sistema OK (exit 0) vs. Sistema con issues: [lista] (exit 1).
 - Si el auto-sync falla: aparece X index — auto-sync falló o auto-sync timeout. Esto sí es un fallo real — el script no reintenta. Acción manual:

`python3 vantage.py sync`

desde `Layer_1/scripts/`, y verificar con `vantage.py status` que `entities_after` $\geq$ `entities_before`.

Tickets pendientes: se listan explícitamente solo CRÍTICO y ALTO; MEDIO/BAJO/Sin Prioridad aparecen solo como conteo — ver Notion para detalle.

Sync manual sigue disponible para forzar fuera de umbral, o si has realizado cambios masivos en el Tracker o el Canon y no quieres esperar a la siguiente corrida de start:

`python3 vantage.py sync`

¿Qué es el Census ID?

Es tu mapa de navegación — te dice en qué documento y en qué bloque exacto vive cada ID del sistema, con link directo. Pero es un mapa, no el territorio: si el Kernel cambia y el Census no se actualiza, el mapa miente. Este es el mismo Census que se verifica y actualiza durante el Ciclo de Sesión (MANUAL:SESSION-CYCLE).

¿Cuándo se regenera?

- Antes de marcar cualquier ticket como cerrado, si ese ticket cambió el estado de un ID (de pendiente a resuelto, o creó uno nuevo).
 - Si no tienes Terminal a la mano en ese momento, el ticket se queda en Blocked-Census — no se cierra en falso, se marca como bloqueado hasta que puedas correr el script.

¿Cómo corre?

```
source ~/Documents/03 Projects/VANTAGE/Layer_1/.venv/bin/activate
cd ~/Documents/03 Projects/VANTAGE/Layer_1/scripts
python3 generate_census.py
```

El script también detecta IDs que existen en los documentos pero no en su lista de seguimiento (“huérfanos”) y te los señala — ya no se cuelan en silencio. Y para cada ID resuelto genera el link exacto al bloque en Notion, no solo al documento. Orden con Changelog:

- Primero Census actualizado
 - Después la entrada de Changelog.
 - Nunca al revés (esto es exactamente el paso 2 del Cierre de Sesión, MANUAL:SESSION-CYCLE).
 - Al cerrar sesión: si hubo cambios a documentación o bases de datos, se te presenta automáticamente un resumen de lo que quedó hecho vs. pendiente — sin que tengas que pedirlo.
 - Aviso en arranque: health_check.py (alias start) reporta la antigüedad del Census en cada corrida — ! census — Nd sin regenerar si pasó el umbral de 7 días. Es solo un recordatorio visual, no dispara nada automáticamente; sigue siendo tu responsabilidad correr generate_census.py cuando cierres un ticket que cambió estado de un ID.

Aplicación de Hipervínculos Cross-Reference

Con el Census ya resolviendo cada ID a su bloque real, el siguiente paso es convertir esas menciones en links clickeables — eso lo hace apply_hyperlinks_notion.py, no generate_census.py (el Census resuelve, el script de hipervínculos escribe).

Cuándo correrlo: después de cualquier alta/baja/rename de ID canónico, o tras una migración de formato de heading — en ambos casos el mapping interno puede

quedar desactualizado.

Cómo corre:

```
python3 apply_hyperlinks_notion.py --all --apply
```

Escribe PATCH puntual directo sobre bloques de Notion, preservando block-ID — nunca pasa por destroy/rebuild (ver KERNEL:ARCHITECTURE-L4 para el riesgo que esto evita). Sin --apply corre en modo dry-run, reportando cuántos hipervínculos propuestos hay por documento sin escribir.

apply_hyperlinks.py (variante anterior, sobre .md locales) queda DEPRECATED — no usar para escrituras nuevas.

Después de correr --apply sobre un documento, evita correr vdoc local sobre ese mismo documento hasta la siguiente sesión (ver riesgo de destroy/rebuild, KERNEL:ARCHITECTURE-L4) — de lo contrario los anchors recién escritos quedan huérfanos.

Skills de Mantenimiento del Tracker (VANTAGE)

Con el Census y su ciclo de regeneración ya cubiertos arriba, esta es la contraparte operativa del lado de Bug/Task Tracker y Changelog: cinco skills que Claude ejecuta bajo invocación explícita del operador, cada una con su propio contrato de Dry Run + APROBAR_WRITE cuando corresponde.

- vantage-create-bug-task — crea un ticket nuevo en Bug Tracker (defecto reactivo) o Task Tracker (trabajo/decisión pendiente). Úsala en cualquier momento de la sesión en que detectes o reportes uno de los dos.
 - vantage-tidy-bug-task-tracker — archiva tickets ya resueltos (confirmación directa del operador, o detección indirecta vía Change Log). Requiere Dry Run + APROBAR_WRITE antes de archivar.
 - vantage-tidy-opportunities-tracker — identifica duplicados y vacantes expiradas en el Tracker de vacantes para archivado, usando los mecanismos de fingerprint y protección de estado terminal ya implementados en feed_processor.py (ver MANUAL:DATA-MANAGEMENT). Requiere Dry Run + APROBAR_WRITE.
 - vantage-tidy-changelog — mantiene el Change Log con las últimas 10 entradas visibles, moviendo el exceso al Archivo Changelog histórico. Úsala cuando el Change Log activo supera 10 entradas o para housekeeping documental puntual.
 - vantage-present-handoff — genera el snapshot de contexto de sesión para continuidad en un chat nuevo. Es independiente y se puede invocar en cualquier momento; no requiere que la sesión esté cerrando (ver también MANUAL:SESSION-CYCLE, Cierre, donde vantage-session-close la invoca automáticamente como parte de su secuencia normal).
 - Cada una declara su propio verbo de apertura/cierre (KERNEL:SKILL-ANNOUNCE-CONVENTION) — nunca el lenguaje de Bootstrap ni de Session Ledger.

Problemas Comunes y Soluciones

Pipeline No Corre

- Verificar .env en ~/vantage_notion_audit/.
 - Confirmar token Notion no expirado (regenerar en Notion → Settings → API → New token).
 - Verificar entorno Python activo: source Layer_1/.venv/bin/activate && python --version.
 - Confirmar permisos de ejecución: ls -la ~/vantage_pipeline.sh (debe tener x).

Entity Index Desactualizado

- Desde v8.7.6: health_check.py detecta índices >24h y dispara vantage.py sync automáticamente en cada corrida de start — no requiere acción manual en el flujo normal.
 - Síntoma de que el auto-sync falló: health_check.py reporta × index — auto-sync falló o auto-sync timeout en vez del ✓ esperado.
 - Solución manual (solo si el auto-sync falló): python vantage.py sync desde Layer_1/scripts.
 - Verificar resultado: vantage.py status debe mostrar entities_after >= entities_before.
 - Si persiste: verificar token Notion y conectividad a internet.

L3 No Procesa Correos

- Verificar layer_3.env existe en Layer_3/config/.
 - Confirmar credenciales: IMAP (Gmail), GROQ_API_KEY.
 - Ejecutar manualmente: vl3 (debe procesar hasta 10 correos).
 - Revisar heartbeat: cat ~/.vantage/l3_heartbeat.json (última ejecución exitosa).
 - Si falla autenticación IMAP: regenerar app password de Gmail.

12.1 MANUAL:FIGMA-SYNC-DIAGNOSTIC

Matriz de Errores — Figma Sync

"Keys sin resolver" tiene dos causas distintas:

- Causa A — Registry desincronizado: la KEY no existe en REGISTRY/registry_seed.json (Skeleton cambió o se agregó sección sin re-mapear). Se arregla actualizando registry_seed.json.
 - Causa B — Nodo ausente en el lienzo activo: el ID sí está en el registro, pero el nodo TEXT fue borrado, movido o convertido a otro tipo en el archivo abierto. Se arregla restaurando el nodo — nunca renombrando capas, porque el plugin ignora nombres y solo usa IDs crudos.
 - Checklist de situaciones:
 - | Situación | Diagnóstico | Acción |
 - | --- | --- | --- |
 - | "0 nodos actualizados" | Regex no detectó ningún bloque figma_text_id válido | Revisar el patrón exacto de MANUAL:FIGMA-SYNC-002 |

- | "Fricción Estructural: IDs duplicados" | Misma KEY 2+ veces en el .md | Eliminar el bloque repetido antes de reintentar |
- | "Keys sin resolver: N" (secciones nuevas) | Causa A — KEY no existe en Registry | Actualizar registry_seed.json o usar solo keys del Golden Skeleton vigente |
- | "Keys sin resolver: N" (keys que ya existían) | Causa B — nodo borrado/movido/reagrupado | Restaurar el nodo TEXT original, sin cambiar su ID |
- | Se pierde la negrita esperada | Falló carga de fuente Bold/Medium en cascada | Verificar variantes Bold/Medium instaladas en Figma |
- | Cursivas o links desaparecen | Sanitización esperada, no un fallo | Ver MANUAL:FIGMA-SYNC-004 — solo negrita sobrevive |

Dashboard No Abre

- Verificar Flask corriendo: Isot -i :8000 (debe mostrar proceso Python).
 - Ejecutar smoke test: vd debe imprimir "SMOKE PASSED — abriendo dashboard".
 - Si falla smoke test: revisar dashboard_start.sh permisos (chmod +x).
 - Verificar puerto 8000 libre: killall -9 Python si hay proceso zombie.
 - Si error de importación: confirmar .venv activo y dependencias instaladas.

REVIEW_NEEDED No Se Resuelve Tras Corregir

- Confirmar que cambiaste Status → Target en Notion (no otro valor).
 - Verificar que corrección se guardó (refrescar página Notion).
 - Ejecutar pipeline: ~/vantage_pipeline.sh.
 - Si persiste: verificar en terminal qué campo sigue bloqueando (Python imprime razón).
 - Revisar logs en ~/.vantage/logs/ para diagnóstico detallado.

vl1 batch Modifica Entradas Sin --execute

- Bug crítico: reportar inmediatamente.
 - Workaround: verificar siempre con vl1 batch (sin flag) antes de ejecutar.
 - Confirmar que script tiene guard if not args.execute: return al inicio.

vsync_doc.py Falla — "blocks.children.list() returned None"

- Bug conocido de notion-client 3.x.
 - Solución: vsync_doc.py usa safe_list() con httpx directo (3 reintentos).
 - Si persiste: verificar que page_id sea válido y token tenga permisos de lectura.
 - Alternativa temporal: sync manual vía MCP Notion.

Score = 0 en Vacante Que Parece Relevante

- Verificar que URL esté activa (no 404/403).
 - Confirmar que JD contenga keywords VM (Python busca términos específicos).
 - Revisar VM_Scope asignado (debe ser Core/Adjacent, no Off-Target).
 - Si todo está correcto: revisar pesos de scoring en profile_config.yaml.

- No modificar Score manualmente (campo Class B, Python lo recalcula).

Gate = BLOCKED Recuperable Pero el Dashboard No lo Detecta

- Confirmar que entrada aparece en dropdown del Dashboard.
 - Verificar que Gate_Decision = BLOCKED (no EXPIRED ni vacío).
 - Si no aparece: refrescar cache de Runtime (vantage.py sync).
 - Si aparece pero validación falla: revisar logs de run_pipeline.py en Dashboard.
 - Filosofía de fallo: [KERNEL:FAIL-PHILOSOPHY](V | KERNEL) (ver también MANUAL:FAILURE-PHILOSOPHY de este Manual).
 - Reglas de Oro: KERNEL:CV-GOLDEN-RULES (ver también MANUAL:CV-GOLDEN-RULES-INDEX de este Manual).
 - Schema de datos: KERNEL:SCHEMA.
 - Gate Decisions: KERNEL:GATE-DECISION (ver también MANUAL:HOW-IT-WORKS de este Manual).

——13 MANUAL:PROMPTS-WRAPPERS

Prompts & Wrappers

Se consultan desde la PROMPT LIBRARY en Notion vía el Weekly Prompt Assembler (weekly_prompt_assembler.py, alias vasseble) — el script hace fetch de cada componente por su ID, sustituye fecha y concatena localmente; el operador recibe archivos .md ya ensamblados, listos para pegar en cada motor, según el orden descrito en MANUAL:WEEKLY-FLOW-001. El catálogo se organiza en tres grupos:

- Prompt A + Wrappers — el Prompt Base (perfil, reglas de exclusión) combinado con el Wrapper específico de cada canal: Career Sites, LinkedIn, Aggregators, Gemini, Grok, you.com.
 - Prompts B y C — búsqueda complementaria, utilizables en cualquiera de los tres motores L2 (Gemini, Grok, you.com).
 - Prompt E — consolidación final; no se concatena con Prompt A, se usa solo tras compilar los JSONs de L1+L2.

14 MANUAL:LAZY-LOAD

Lazy Load

Cómo la IA lee el KERNEL y el CAREER CANON (Lazy Load)

La extracción de reglas y contratos lógicos (Lazy Load) opera con la siguiente prioridad:

Prioridad A — Terminal (canónico): lazy_loader.py ejecuta Server-Side Lazy Load. Parsea bloques hijos de la Notion API y devuelve únicamente el payload del ID solicitado. Consumo: ~150 tokens por llamada.

Prioridad B — MCP Notion: reservado exclusivamente para escrituras (APROBAR_WRITE) y modificaciones estructurales de páginas. No se usa para lectura de reglas o contratos.

15 MANUAL:PATCH-QUALITY

Calidad de Parches

Todo parche a los 6 documentos fundacionales debe cumplir estos seis criterios antes de aplicarse — si falla alguno, se reescribe antes de solicitar

APROBAR_WRITE:

1. Invisibilidad estructural — no crea secciones nuevas si el contenido cabe en una existente. Nota: la invisibilidad estructural incluye el nivel de heading Markdown, no solo el contenido — una subsección (NN.N) que comparte nivel ## con su capítulo padre rompe esta invisibilidad tanto como un párrafo con tono distinto. Ver la matriz tipográfica congelada en KERNEL:DOCUMENTATION-001 como referencia de nivel correcto por tipo de nodo. Adicionalmente, el identificador técnico y el título descriptivo de cualquier nodo deben coexistir dentro de un único bloque de heading (un solo nodo ##/###), con el título unido al identificador mediante un salto de línea
2. interno al mismo bloque — nunca como dos bloques de heading consecutivos, aunque visualmente ambos casos puedan parecer "dos líneas" a simple vista. El espaciado visual que Notion aplica entre dos bloques de heading consecutivos en su renderizado es un artefacto de la plataforma, no una instrucción para insertar contenido de separación — esta regla aplica a todo nodo del sistema documental, no a un caso puntual.
3. Continuidad de voz — mismo registro y nivel técnico del bloque que lo rodea.
4. Progresión narrativa intacta — el lector no debe notar un salto temático al leer de corrido.
5. Diff mínimo — se edita solo el texto indispensable, nunca el bloque completo si un párrafo basta.
6. Coherencia transversal — no puede contradecir ni duplicar una definición ya existente en Kernel, System Prompt, Career Canon o Aliases.
7. Concreción de títulos — el título descriptivo de cualquier nodo debe ser ilustrativo y concreto (una palabra o frase corta que nombra el contenido), no una construcción semántica compuesta o explicativa. Ejemplo de dirección: preferir el nombre directo del tema sobre una paráfrasis de su función.
8. Un parche que pasa estos seis filtros no se distingue, seis meses después, del texto que rodeaba su punto de inserción original.

——16 MANUAL:GOLDEN-RULES

Reglas de Oro

Base: KERNEL:CV-GOLDEN-RULES.

El contenido detallado de estas reglas vive en el Kernel del sistema y no está reproducido en este Manual más allá de esta referencia. Ver lista de huecos detectados al final de este documento.

——17 MANUAL:SLA

SLA de Latencia

Nota: el SLA "< 45 minutos" cubre únicamente el segmento Score calculado → Ready-to-Apply (Discovery → Ready-to-Apply en nomenclatura anterior). El segmento Trigger → Score depende del ciclo de ejecución de ~/vantage_pipeline.sh (ver MANUAL:WEEKLY-FLOW-001, Lunes) — no tiene SLA fijo salvo ejecución manual explícita de layer_1_run.py.

18 MANUAL:CV-GOLDEN-RULES-INDEX Reglas de Oro CV

Las Reglas de Oro (KERNEL:CV-GOLDEN-RULES) son restricciones de arquitectura, no preferencias. Viven íntegras en el Kernel — esta sección es un índice de navegación, no una copia.

ID	Regla	Qué bloquea
KERNEL:CV-GOLDEN-RULES-001	No Evaluar Fit Antes de Escribir	Preguntas de "¿me conviene esta vacante?" — el fit lo decide Score (Python) + el operador
KERNEL:CV-GOLDEN-RULES-002	No Calcular ni Estimar Campos Class B	Estimar Score, Gate_Decision, VM_Scope, etc. — son Python-only
KERNEL:CV-GOLDEN-RULES-003	No Cuestionar la Calidad de Datos del Usuario	Comentarios sobre volumen/calidad del JSON de búsqueda — estrategia es 100% humana
KERNEL:CV-GOLDEN-RULES-004	No Delegar Escritura al Usuario	"Copia esto y pégalo en Notion" — el sistema escribe directo, salvo export PDF/Drive
KERNEL:CV-GOLDEN-RULES-005	No Interpretar en SYNC	SYNC reporta datos puros, sin análisis ni recomendaciones
KERNEL:CV-GOLDEN-RULES-006	Invarianza del Gate	Prohibido que la IA re-evalúe o bloquee vacantes ya aprobadas

- Toda violación produce el Template Universal de Rechazo (ver Kernel):
OPERACIÓN RECHAZADA → razón → alternativa operativa → confirmación SÍ/CANCELAR.
 - Para el detalle completo de cada regla (ejemplos de solicitudes que la activan, redacción exacta de la respuesta estandarizada), consultar directamente KERNEL:CV-GOLDEN-RULES en el Kernel — fuente única,

no se replica aquí para evitar drift entre documentos.

19 MANUAL:POSITIONING-CRITERIA

Positioning Criteria
 CANON:POSITIONING define 4 modos de posicionamiento para CV-B. Esta sección resuelve el gap operativo: con qué criterio elegir uno.

Modo	ID	Ancla canónica	Cuándo aplica	Señales de Alarma
N1	CANON:POSITIONING-001	C01 · 3 marcas lujo · CAPEX/OPEX · NPI	JD enfatiza gestión multi-marca de lujo, presupuesto, lanzamientos de producto	No usar si el JD es retail masivo/fast fashion o venta por volumen, sin componente multi-marca de lujo
N2	CANON:POSITIONING-002	C02 · Adidas Brand Center · KPI07 · blueprints	JD enfatiza Store Design, Flagship, construcción / remodelación física	No usar solo por "diseño" — requiere obra física, planos, coordinación con arquitectos, no solo estética
N3	CANON:POSITIONING-003	C03 · 270+ POS · 6 países · KPI03–06 · CF05	JD enfatiza rollout regional multi-país, estandarización, eficiencia operativa	No usar si el JD es de un solo mercado/tienda — sin alcance multi-país no hay ancla regional real

N4	CANON:POSITIONING-004	C04/C05 · +43% tráfico · +18% conversión · 21 reportes	JD enfatiza liderazgo de campo comercial, KPIs de tráfico/ conversión, gestión de equipos directos	No usar si el JD no reporta KPIs comerciales medibles (tráfico/ conversión) o no involucra equipos de campo
----	-----------------------	--	--	---

- Regla de desempate (JDs híbridos) — ver CANON:POSITIONING para el texto completo: (1) más keywords mapeados al ancla, (2) empate → mayor seniority (N2>N1, N4>N3 con presupuesto regional explícito), (3) empate persistente → escalar a decisión humana vía fit_gaps.
 - Gestión de Ambigüedad — JDs Híbridos
 - Cuando el algoritmo de KERNEL:CV-PIPELINE-001 escala a decisión humana (empate persistente), fit_gaps del HANDOFF debe declarar el conflicto explícitamente: qué dos modos empataron, cuántos keywords mapeó cada uno, y qué falta para resolverlo sin intervención humana (ej. "N1 vs N2 empatados 3-3 — JD menciona lujo y construcción de flagship en igual peso; falta señal de presupuesto regional para desempatar por seniority"). CV-A no entrega HANDOFF completo sin esta nota cuando aplica el escalamiento.

20 MANUAL:GOLDEN-SKELETON-REF

Figma Sync & Golden Skeleton

El "Golden Skeleton" (CANON:OUTPUT-CONTRACT-002) es la secuencia fija de bloques ##### figma_text_id que todo CV-B debe replicar exactamente — mismo conteo, mismo orden, solo cambia el contenido textual.

- SSOT de IDs de nodo Figma: registry_seed.json en 04-Vantage_CV/Figma Sync/.
 - Slots clave: 2055:9 (Nombre), 2055:10 (Tagline), 2043:51 (Perfil), 2043:56-60 (Skills), 2043:64+ (Experiencia).
 - Regla de invariancia: si el Skeleton cambia en Figma, registry_seed.json se actualiza antes del siguiente CV-B — nunca al revés.
 - Detalle completo del protocolo (immutability, slot integrity, null-fill rule) vive en CANON:OUTPUT-CONTRACT-002 — no se replica aquí.

20.1 MANUAL:FIGMA-SYNC-001

Arquitectura del Ecosistema

Tres piezas:

- manifest.json (identidad y permisos mínimos, sin red)
 - ui.html (parser/UI)
 - code.js (Registry V2 + escritura de nodo). Comunicación exclusiva vía

postMessage entre ui.html y code.js — ver KERNEL:ARCHITECTURE-L4 para el diagrama de flujo completo.

20.2 MANUAL:FIGMA-SYNC-002

Contrato de Bloque

Cada slot del .md debe seguir el patrón estricto marcado en el Golden Skeleton: encabezado ##### figma_text_id(KEY) seguido del contenido con negrita donde aplique. Sin esa cabecera exacta (nivel de heading correcto seguido del KEY entre paréntesis), el bloque no se detecta — el síntoma es "0 nodos actualizados" sin error explícito.

20.3 MANUAL:FIGMA-SYNC-003

Flujo de Inyección

Cuatro fases, en orden:

1. Detección — ui.html revisa si el texto contiene figma_text_id; si no, intenta JSON.parse.
2. Parsing — regex extrae cada bloque, detecta KEYs duplicadas (detiene el envío si las hay) y sanitiza contenido.
3. Resolución — code.js busca cada KEY en REGISTRY/registry_seed.json; si no está, la intenta usar como ID crudo directo. Búsqueda O(1) vía figma.getNodeById — nunca por nombre de capa.
4. Escritura — reemplazo total del texto del nodo (node.characters), con negrita quirúrgica por rango si aplica.

20.4 MANUAL:FIGMA-SYNC-004

Sanitización de Contenido

Sobrevive: negrita (rango tipográfico real en Figma).

Se elimina sin aviso: cursiva con guion bajo, links (se conserva solo el texto visible), backslashes de escape.

Sin tratamiento especial: bullets con guion o asterisco, numeración, tablas — el "bullet" real ya lo define la estructura del Golden Skeleton, no la sintaxis Markdown.

20.5 MANUAL:FIGMA-SYNC-005

Regla de Reemplazo Total

La inyección es node.characters = item.text — reemplazo total, no merge ni append. Cualquier edición manual hecha directamente en Figma sobre esos nodos se sobrescribe sin aviso ni backup en el siguiente sync. La única fuente de verdad post-sync es el .md pegado, no el estado previo del lienzo.

21 MANUAL:SCHEMA-FIELD-REF

Schema Class A/B

KERNEL:SCHEMA-001 define ownership exclusivo por campo. Esta tabla es índice de consulta rápida — el contrato completo (reglas de excepción, mapeo de vocabulario) vive en el Kernel.

Class A — Human-Primary (operador/feed_processor escriben):

Rol · Marca · Source_Type · URL · Status · Positioning_Mode ·

Prioridad · Holding · JD · NAD · layer · hash

Class B — System-Primary (Python únicamente, ningún otro componente escribe):

Score · Gate_Decision · VM_Scope · Role_Class · Match ·

Next_Action · Fetch · Fuente

Fetch refleja verificación técnica real, incluso en agregadores — un valor Accesible ya no puede escribirse sin al menos un intento de request.

**Next_Action: select (10 valores operativos). Ver
KERNEL:SCHEMA-008.**

Excepción documentada: Fuente_Manual (Class A) existe para valores de fuente que deben persistir entre runs — Fuente (Class B) se sobrescribe en cada corrida (KERNEL:SCHEMA-003).

Pesos de Score/VM_Scope: viven en profile_config.yaml, propiedad de Python — el Manual no reproduce los valores numéricos porque son deuda de implementación, no contrato documental (ver KERNEL:GATE-DECISION-002). Un operador que necesite ajustar pesos debe editar ese archivo directamente, no este documento.

Verificación de Longitud (--length / --update-baseline)

Contexto:

Procedimiento de sanity check para validar que ningún documento fundacional haya sufrido pérdida silenciosa de contenido tras operaciones de sincronización, edición masiva o fallo en scripts de inyección.

Procedimiento:

1. Ejecución básica de verificación (read-only):

```
python vversions --length
```

- Salida esperada: Tabla detallada con veredicto por documento y veredicto final.

- Exit code: 1 si el veredicto final es ATENCIÓN REQUERIDA.

1. Actualización de baseline tras edición legítima (write):

```
python vversions --length --update-baseline
```

- Precondición: Confirmar previamente que las variaciones de líneas son esperadas (ej. adición de secciones o refactorización documentada).
 - Efecto: Sobrescribe length_baseline.json con las métricas actuales y actualiza el timestamp captured_at de todos los documentos.
 - Casos de uso clave:
 - Sanity check pre-sync: Ejecutar siempre antes de sincronizar cambios masivos.

- Post-incidente: Diagnóstico rápido ante sospechas de truncamiento por falla de API o timeout.

——22 MANUAL:SCRIPT-GLOSSARY

- feed_processor.py:
 - should_mutate_existing_page(page, schema): Usado por _set_dedup_flag_if_needed y _upgrade_layer_if_needed para validar mutación según el guard de KERNEL:GATE-DECISION-012.
 - profile_fit.py:
 - should_annotate_existing(status): Determina si un registro existente puede ser anotado (Dedup_Flag/layer) según su Status.
 - Glosario de Scripts — Referencia Operativa en Humano

Propósito: traducir cada script/wrapper del árbol activo (Layer_1, Layer_3, Layer_4, Dashboard, Raycast) a lenguaje operativo — qué hace, por qué existe, y un caso de uso concreto por cada flag disponible. Este documento es el "manual de instrucciones" legible; la Script Library (Notion DB) es su índice corto y estructurado. Se referencian entre sí — ver MANUAL:SCRIPT-GLOSSARY-XREF al final.

Nodo padre de [KERNEL:DOMAIN-ARCHITECTURE] — organizado por capa, no alfabéticamente, para que la lectura secuencial siga el flujo real del sistema (L1 → L3 → L4 → Dashboard → Raycast).

——22.1 MANUAL:SCRIPT-GLOSSARY-L1

Layer 1 — Active Recon & Core Pipeline

22.2 MANUAL:SCRIPT-GLOSSARY-CV-PREPCV Pipeline — Preparación Mecánica (Miércoles)

Contraparte operativa de KERNEL:CV-PIPELINE-003 — los 3 scripts que generan el HANDOFF scaffold en batch antes de la sesión de Claude. Ver también MANUAL:WEEKLY-FLOW-003 para cuándo usar esta ruta vs. copiar URL/JD manualmente.

adapt_tracker_export.py

Qué hace: normaliza el export crudo de Notion (CSV) al formato de cv_a_batch_agent.py — renombra columnas, externaliza JDs, deduplica, valida.

Flags:

Flag	Caso de uso
--in <ruta> (requerido)	CSV export crudo de Notion.
--out-dir <ruta>	Directorio de salida (default ./adapted).
--skip-dup-check	Cuando ya sabes que el export no trae duplicados.
cv_a_batch_agent.py	

Qué hace: orquesta cv_a_prep.py en paralelo sobre filas con Next_Action=Optimizar, con checkpoint y reintentos.	
Flags:	
Flag	Caso de uso
---	---
--csv <ruta> (requerido)	tracker_adapted.csv de adapt_tracker_export.py.
--workers <N>	Workers paralelos (default 3).
--resume	Retoma desde el último checkpoint tras interrupción.
--no-progress	Desactiva barra de progreso (logs no interactivos).
cv_a_prep.py	
Qué hace: preparador individual — cache de JD (24h), detección de idioma, Hard Block check, genera el scaffold.	
Flags:	
Flag	Caso de uso
---	---
--url / --jd-file	Fuente de la vacante.
--empresa / --rol	Metadata opcional pre-llenada.
--out <ruta>	Ruta de salida del scaffold.
--clear-cache	Limpia cache expirada antes de correr.

——22.1 MANUAL:SCRIPT-GLOSSARY-L1Layer 1 — Active Recon & Core

Pipeline

layer_1_run.py

Variables de entorno (tuning silencioso):

Variable	Default	Caso de uso
----------	---------	-------------

ENABLE_DEDUP_AUDIT	true	La auditoría de duplicados corre automáticamente al final del pipeline; usa false solo para desactivarla en una ejecución puntual. El subproceso hereda --dry-run y opera con timeout de 10 minutos.
DEDUP_WINDOW_DAYS	60	Ajusta la ventana temporal enviada al audit de dedup sin modificar código.
Flag	Caso de uso	
---	---	
--dry-run	Antes de correr el pipeline completo en un día con muchos feeds nuevos, corre con --dry-run para ver qué escribiría sin comprometer el Tracker — útil si sospechas que un feed trae datos sucios.	
--dedup-audit	Al cerrar el ciclo semanal de L1, agrégalo para que el mismo comando dispare dedup_opportunities.py como subproceso y te dé el reporte fuzzy sin correr dos comandos separados.	

generate_archive_notes() — función interna, sin CLI propia. Invocada desde 3 puntos de layer_1_run.py (URL Gate, misfit de perfil, NAD vencido) para escribir la nota determinista de archivado en Notas (ver KERNEL:GATE-DECISION-013, MANUAL:DATA-MANAGEMENT-001). Mecanismo de herencia de --dry-run pendiente de verificar contra código fuente.		
feed_processor.pyQué hace: Ingiere un JSON de feed (L1/L2/L3) y crea/actualiza registros en el Tracker.		
Flags:		
Flag	Caso de uso	
---	---	
--file (requerido)	Siempre necesario — apunta al JSON del feed a procesar. Rutas relativas se resuelven desde Layer_1/.	

--layer {1,2,3}	Si estás cargando un feed que viene de investigación manual en Perplexity (no de L1 automatizado), usa --layer 2 para que el Tracker lo etiquete correctamente como fuente estratégica.	
--fast	Encontraste UNA vacante urgente fuera de tu ciclo semanal (ej. alguien te la compartió por WhatsApp) — usa --fast para meterla sola sin esperar al batch de los lunes. Rechaza feeds con más de un item.	
--interactive	Cuando el feed trae vacantes de calidad mixta y quieres decidir una por una ([S]í/[O]mitir/[Q]uit) en vez de que todo se escriba automáticamente. Ojo: si eliges Quit a medio camino, lo ya escrito no se revierte.	

generate_census.py Qué hace: Genera el ID Census — barrido completo de IDs canónicos en los 9 documentos fundacionales, detecta huérfanos, y puede corregirlos o sincronizarlos a Notion sin intervención manual.		
Flags:		
Flag	Caso de uso	
---	---	
--debug-id <id1> <id2> ...	Ya sabes que KERNEL:SCHEMA-008 está fallando en el census y no quieres esperar el barrido completo — pásalo directo y te da diagnóstico quirúrgico de esos IDs específicos.	
--auto-fix-orphans	El census te reportó 40 huérfanos y no quieres darlos de alta uno por uno en CENSUS_SPEC a mano — corre esto y te los agrega interactivamente con el comentario de auditoría ya insertado.	

--sync-to-notion <page_id>	Ya corriste el census y quieres que el export quede reflejado en el V-ID-CENSUS de Notion sin copiar/pegar manualmente — pásale el page_id destino.	
generate_entity_index_v2.py Qué hace: Reconstruye el índice de entidades (entity_index_v2.json), el grafo de relaciones y los backlinks — la base de datos interna que usa vantage.py ask/query.		
Flags:		
Flag	Caso de uso	
---	---	
--limit <N>	Estás probando un cambio en la lógica de indexado y no quieres esperar a que procese todas las fuentes — limita a N entidades por fuente para iterar rápido.	
--out <ruta>	Quieres generar un índice de prueba sin pisar el archivo real que usa producción — apunta a una ruta temporal.	

--skip-graph	Solo necesitas refrescar el índice de entidades (para vantage.py query) y no te importa el grafo/backlinks en este momento — ahorra tiempo de corrida.	
generate_id_inventory.py Qué hace: Escanea un árbol de archivos y genera un inventario CSV/Markdown de todas las definiciones y referencias de IDs canónicos encontradas.		
Flags:		
Flag	Caso de uso	
---	---	
--root <ruta> (requerido)	Quieres auditar solo una subcarpeta (ej. solo Layer_1/) en vez de todo VANTAGE — acota el escaneo.	
--out <ruta>	⚠ Nota real: el default NO es ./out como dice el help — es una ruta absoluta fija a tu carpeta Layer_1/data. Si corres esto desde otra máquina o usuario, especifica --out explícitamente o el inventario se va a una ruta que no existe.	

apply_hyperlinks_notion.py Qué hace: Aplica el sistema de cross-reference hyperlinks entre documentos fundacionales (PATCH de bloques en Notion).		
Flags:		
Flag	Caso de uso	
---	---	
--doc {kernel,system_prompt,manual,career_canon,aliases,change_log,brief}	Acabas de editar solo el Manual y no quieres re-procesar los 7 documentos — corre el ciclo de hyperlinks en uno solo.	
--all	Ciclo completo de housekeeping documental — mutuamente excluyente con --doc.	
--apply	El modo real de escritura — sin este flag, cualquier corrida (incluso sin --dry-run) es preview únicamente.	
--dry-run	⚠ No tiene efecto propio — el script ya es dry-run por default sin --apply. Es no-op explícito, no un modo adicional.	
normalize_heading_ids.py Qué hace: Audita y corrige headings legacy o mal formados en los documentos fundacionales.		
Flags:		

Flag	Caso de uso	
---	---	
--csv <ruta>	Quieres revisar en Excel/Numbers qué headings están mal antes de decidir si vale la pena corregirlos — exporta el reporte sin tocar Notion.	
--apply	Ya revisaste el CSV y confirmaste que los fixes son correctos — aplica los reemplazos vía API.	
--yes	Vas a correr --apply en un batch grande ya pre-aprobado y no quieres que te pregunte confirmación por cada heading.	
consolidate_duplicates.py	Qué hace: Detecta y fusiona registros duplicados en el Tracker, archivando el sobrante.	
Flags:		
Flag	Caso de uso	
---	---	
--dry-run	Antes de fusionar nada, corre esto para ver qué grupos detectaría como duplicados.	
--yes	Ya revisaste el dry-run y confías en el resultado — salta la confirmación interactiva.	

--aggressive	El matching normal (por URL) no está agrupando vacantes que sabes que son la misma posición republicada con URL distinta — usa matching por marca+rol. No fusiona URLs de vacantes genuinamente distintas.	
cross_tracker_match.py yQué hace: Busca coincidencias entre el Tracker activo y el Archivo Tracker por regla marca+rol.		
Flags:		
Flag	Caso de uso	
---	---	
--dry-run	⚠ Importante: default es True vía store_true, y no existe forma de desactivarlo — este script siempre es solo-reporte, nunca ejecuta acciones. Si tu intención es que alguna vez actúe, hoy no puede — es candidato para el punto B (documentación transversal) si quieres dejarlo explícito en Kernel.	
dedup_opportunities.py		
Flags:		
Flag	Caso de uso	
---	---	

--window-days N	Ajusta la ventana de comparación fuzzy; default 60.	
--dry-run	Simula grupos y marcas candidatas sin escribir Dedup_Flag en Notion.	
--clear <page_id>	Un registro quedó marcado erróneamente como "Posible duplicado" y quieres limpiar solo ese flag sin re-correr toda la auditoría. ⚠ Solo accesible desde Terminal directo — el wrapper de Raycast (dedup_audit.sh) no lo expone.	
Variables de entorno (tuning silencioso):		
Variable	Default	Caso de uso
---	---	---
NOTION_ARCHIVE_DATA_SOURCE_ID	674696fd-94b6-464a-ac1f-64b0cc917e15	Define el data source del Archivo Tracker usado por la auditoría cruzada; solo se modifica si cambia dicho origen.
Modo	Caso de uso	
---	---	
Sin argumentos	Auditoría general — lo que corre dedup_audit.sh/ Raycast.	

--clear <page_id>	Un registro quedó marcado erróneamente como "Posible duplicado" y quieres limpiar solo ese flag sin re-correr toda la auditoría. ⚠ Solo accesible desde Terminal directo — el wrapper de Raycast (dedup_audit.sh) no lo expone.	
backfill_class_a.pyQué hace: Backfill de campos Class A (layer, hash, Prioridad) en registros existentes del Tracker. Contiene su propio hack de parsing local para leer propiedades top-level del objeto Notion (mismo patrón de riesgo que created_time en priority_logic.py, ver KERNEL:TRIGGER-002) — depende de la estructura de chunks del objeto recibido, no de un parser compartido.		
Flags:		
Flag	Caso de uso	
---	---	
--dry-run	Antes de correr un backfill masivo tras un cambio de schema, revisa qué se llenaría.	

backfill_next_action_select.py Qué hace: Migra el campo Next_Action de texto libre a select tipado. — MOVIDO a Archive/Legacy_Scripts/ (saneamiento v9.21.x)		
Flags:		
Flag	Caso de uso	
---	---	
--execute	Ya validaste el dry-run (default) y quieres aplicar la conversión real.	
verify_versions.py (alias vversions) Qué hace: Herramienta central de verificación — versión de los 9 fundacionales, gap-report de scripts/skills, e integridad de longitud documental.		
Flags:		
Flag	Caso de uso	
---	---	
--sync	Acabas de subir la versión en Change Log y necesitas propagarla a los otros 8 documentos con verificación de escritura.	

--bootstrap	Al abrir sesión, genera el dump read-only de Ledger + Changelog + tickets prioritarios — más barato que pedirle a Claude que haga fetch manual de cada uno.	
--scripts	Sospechas que hay scripts nuevos sin registrar en Script Library — corre esto para el gap-report. (Ver también punto D — --new-scripts propuesto).	
--skills	Igual que arriba pero para archivos .skill.	
--length	Antes de una sincronización crítica, si sospechas que algún documento se truncó silenciosamente (edición accidental, corte de API), corre esto como sanity check.	
--update-baseline	Solo después de confirmar manualmente que un cambio de longitud fue una edición legítima (no truncamiento) — actualiza el baseline. Requiere --length.	

clean_script_library_links.py Qué hace: Limpia valores de URL corruptos (http:// mal formados) en Script Library.		
Flags:		
Flag	Caso de uso	
---	---	
--apply	Detectaste links rotos en Script Library tras una migración — aplica la limpieza real. Sin el flag, solo lista candidatos.	
backfill_archive_fingerprint.py Qué hace: Calcula y completa fingerprints de deduplicación en el Archivo Tracker.		
Uso: Sin flags CLI propios — se corre directo, solo requiere NOTION_TOKEN/NOTION_API_KEY en el entorno. — MOVIDO a Archive/Legacy_Scripts/ (saneamiento v9.21.x)		
notion_utils.py Qué hace: Cliente HTTP compartido para todas las llamadas a Notion — caché, rate limiting, reintentos. No se invoca directo salvo para diagnóstico.		

Uso (comando posicional):		
Comando	Caso de uso	
---	---	
metrics	Quieres ver cuántas llamadas a Notion se han cacheado vs. hecho en vivo en la sesión actual.	
clear-cache	Sospechas que estás viendo datos obsoletos de Notion pese a un cambio reciente — vacía la caché local.	
reset-metrics	Reinicia los contadores de métricas para medir una corrida específica desde cero.	
Variables de entorno (tuning silencioso — hoy corren con default sin que lo notes):		
Variable	Default	Caso de uso
---	---	---
NOTION_MIN_INTERVAL	0.35 (segundos)	Si notas que tus corridas de L1 son lentas y no te importa el riesgo de rate-limit, puedes bajar este intervalo.
NOTION_CACHE_TTL	21600 (6h)	Si estás iterando rápido sobre un documento y la caché te da versiones viejas, baja el TTL temporalmente.

NOTION_MAX_RETRIES	3	Súbelo si tu conexión es inestable y ves fallos por timeout en vez de reintentos agotados.
VANTAGE_LOG_LEVEL	INFO	Cambia a DEBUG cuando estés troubleshooting un bug de integración con Notion y quieras ver el detalle de cada request.
NOTION_VERSION	2022-06-28 (la mayoría) / 2025-09-03 (resolver_layer_v1.py)	⚠ Nota real: hay inconsistencia entre scripts — mezclar versiones de API en el mismo flujo puede producir HTTP 400 (ya documentado en KERNEL).
health_check.py	Qué hace: Chequeo de salud del sistema — conectividad, versión, tickets pendientes, estado de git.	
Uso:	Sin flags — se corre directo (via vantage-health.sh en Raycast). Exit code 0 = sano, 1 = issues encontrados (no fallo fatal).	
vantage.py	Qué hace: CLI unificado de consulta en lenguaje natural sobre el índice de entidades.	
Comandos:		
Comando	Caso de uso	

---	---	
ask <texto>	Pregunta abierta en lenguaje natural — une todos los tokens del argumento.	
resolve <id>	Resuelve un ID canónico puntual a su contenido — solo usa el primer token.	
context <id>	Similar a resolve pero trae contexto extendido alrededor del ID.	
query <texto>	Búsqueda estructurada — une todos los tokens.	
status	Estado general del sistema, sin argumentos.	
sync	Reconstruye índice/ grafo/backlinks desde Notion — requiere NOTION_TOKEN. Costoso, úsalo solo cuando sepas que el índice está desactualizado.	
feedback_loop.py	Qué hace: Calcula métricas de efectividad y conversión sobre el Tracker.	
Flags:		
Flag	Caso de uso	
---	---	
--json	Quieres pasar el output a otro script o graficarlo — output máquina-legible en vez del reporte de texto.	

toggle_changelog_archive.py Qué hace: Convierte el formato del Archivo Changelog exportado (toggle blocks Markdown). — MOVIDO a Archive/Legacy_Scripts/ (saneamiento v9.21.x)		
Flags:		
Flag	Caso de uso	
---	---	
--file <ruta> (requerido)	El Markdown exportado desde Notion que quieres convertir.	
--out <ruta>	Default es /tmp/archivo_changelog_toggled.md — especifica otra ruta si quieres conservarlo fuera de /tmp.	
--dry-run	Ver las primeras 80 líneas convertidas sin escribir el archivo completo.	
--apply	⚠ No tiene efecto real — el archivo se escribe siempre que --dry-run esté ausente; este flag es vestigial.	
lazy_loader.py Qué hace: Fetch quirúrgico de una sección específica de un documento de Notion (economía de contexto — evita traer el documento completo).		
Flags:		

Flag	Caso de uso	
---	---	
--page <uuid> (requerido)	La página de Notion sobre la que quieres resolver una ruta.	
--route PREFIX:CLAVE (requerido)	Ej. KERNEL:SCHEMA-008 — trae solo esa sección, no el documento entero. Ideal cuando Claude necesita un solo nodo y no quieres gastar tokens en el resto.	
context_layer.py / query_layer.py Qué hace: Resolución de entidades individuales (context_layer.py entity_id) y búsqueda libre (query_layer.py texto) sobre el índice — motor interno detrás de vantage.py resolve/query.		
Uso: Un argumento posicional cada uno; normalmente no se invocan directo, sino a través de vantage.py.		

——extract_scores.py
 Qué hace: Extrae la distribución de Score del Tracker activo — snapshot rápido de cómo se están calificando las vacantes actuales.
 Uso: Sin flags CLI — se corre directo. Requiere NOTION_TOKEN en el entorno.
 Caso de uso: Antes de ajustar el umbral del Score Gate (gate() en layer_1_run.py), corre esto para ver la distribución real y evitar mover el corte a ciegas.

extract_scores_detailed.py
 Qué hace: Igual que extract_scores.py pero genera histograma detallado en vez de resumen.
 Uso: Sin flags CLI — se corre directo. Requiere NOTION_TOKEN.
 Caso de uso: Cuando el resumen simple no te dice si el problema es concentración en un rango específico (ej. muchas vacantes atoradas en 55-60) — el detalle por bucket sí lo muestra.

source_analytics.pyQué hace: Analiza efectividad de fuentes de discovery (LinkedIn, agregadores, L2, L3, etc.) — cuáles fuentes convierten mejor a Gate_Ddecision=CREATE.

Uso: Sin flags CLI — se corre directo. Requiere NOTION_TOKEN.

Caso de uso: Al revisar si vale la pena seguir invirtiendo tiempo en una fuente de L2 específica (ej. Perplexity vs Grok) — esto te da el dato duro en vez de intuición.

vprint.py / vprint.shQué hace: Imprime rápido las oportunidades con Gate_Ddecision=CREATE — vista de "qué está listo para actuar hoy" sin abrir Notion.

Uso: Sin flags CLI — se corre directo. Requiere NOTION_TOKEN y NOTION_DB_OPPORTUNITIES.

Caso de uso: Primera cosa que corres en la mañana para ver qué vacantes pasaron el gate sin tener que entrar al Tracker completo.

⚠ Hallazgo real (no corregido, solo documentado): vprint.sh invoca python3 .../ VANTAGE/vprint.py — ruta incompleta, le falta Layer_1/scripts/. El script real vive en Layer_1/scripts/vprint.py, no en la raíz de VANTAGE. Si el wrapper falla con "No such file or directory", esta es la causa.

get_vantage_digest.shQué hace: Descarga el "digest" completo del repo VANTAGE desde Gitingest (gitingest.com/raw/mauriciomeyran/VANTAGE) — snapshot de texto plano de todo el árbol, útil para pegar contexto masivo a un LLM externo.

Flags:

Argumento	Caso de uso
output_file (posicional, opcional)	Default VANTAGE_digest.txt. Especifica un nombre distinto si quieres conservar varios snapshots fechados sin sobrescribir el anterior.
Layer 3 — Passive Intake (Gmail)	
layer_3_mail.pyQué hace: Lee correos no leídos de una etiqueta Gmail vía IMAP, extrae vacantes con Groq, las escribe en el Tracker.	
Variables de entorno (todas ajustables sin tocar código):	
Variable	Default
---	---
EMAIL_LABEL	.Jobs
GROQ_MODEL	llama-3.1-8b-instant
GROQ_MIN_DELAY_SEC	12
GROQ_MAX_RETRIES	8

GROQ_BODY_MAX_CHARS	3500
GROQ_MAX_EMAILS_PER_RUN	10

——22.1a MANUAL:SCRIPT-GLOSSARY-L1-MODULES

Módulos Compartidos (sin CLI propia — se importan, no se ejecutan solos)

Estos 7 no son "scripts" en el sentido operativo — son librerías internas que otros scripts importan. --new-scripts los detecta igual porque no distingue tipo de archivo; se documentan aquí por completitud, sin tabla de flags porque no tienen ninguno.

gate_logic.pyQué hace: Terminal State Protection (KERNEL:GATE-DECISION-010) — decide si un registro del Tracker ya está en estado terminal (Postulado, Rechazado, Expirada) y por lo tanto NO debe recalcularse. Contiene cero lógica de scoring.

Quién lo consume: layer_1_run.py (Fase 4).

Por qué te sirve saberlo: si un registro se queda "atorado" sin actualizar pese a nueva información, este es el módulo que decide si eso es correcto (protección de terminalidad) o un bug.

class_b_guard.pyQué hace: Guard técnico para GAP-03 (KERNEL:GATE-DECISION-003) — punto único de verdad de qué campos puede escribir un actor no-Python (como Claude vía MCP) en el Tracker. No ejecuta la escritura, audita el payload antes y devuelve la versión limpiada + reporte de lo removido.

Por qué existe: feed_processor.py ya filtra Class B por construcción, pero el conector MCP de Claude escribe directo a Notion sin pasar por ahí — esta es la asimetría que cierra.

Por qué te sirve saberlo: si alguna vez ves un campo Class B escrito por mí (Claude) que no debería, este es el módulo que falló o que faltó invocar.

priority_logic.pyQué hace: Lógica de Prioridad (Class A) compartida — extraída de backfill_class_a.py para romper un import circular con layer_1_run.py.

Quién lo consume: layer_1_run.py (Fase 3.6, escritura primaria semanal) y backfill_class_a.py (catch-up de registros legacy).

Por qué te sirve saberlo: si cambias la matriz de urgencia/importancia que define Prioridad, este es el único archivo que debes tocar. Nota: esto aplica solo a la matriz de Prioridad — la función auxiliar txt() (lectura de propiedades) sí existe duplicada en tres archivos (layer_1_run.py, priority_logic.py, backfill_class_a.py); consolidación evaluada y descartada por riesgo de import circular (mismo motivo por el que existe este módulo compartido).

profile_fit.pyQué hace: Reglas de fit de perfil VM y exclusiones compartidas — detecta títulos de rol excluidos (vendedor, sales, planner sin "visual", store manager, etc.) vía regex.

Quién lo consume: pipeline principal y scripts de cleanup.

Por qué te sirve saberlo: si una vacante que debería excluirse se está colando (o viceversa, una válida se excluye), este archivo tiene el patrón regex responsable — no busques la lógica en otro lado.

graph_layer.pyQué hace: Carga graph_v2.json y backlinks_v2.json, expone funciones de consulta sobre el grafo de entidades (get_archived_from,

get_backlinks, graph_stats).

Quién lo consume: agent_api.py (y por extensión, vantage.py ask).

Por qué te sirve saberlo: si vantage.py ask devuelve relaciones incorrectas o desactualizadas entre entidades, corre vantage.py sync para regenerar los JSON que este módulo lee — no es un bug del módulo en sí.

runtime_identity.py Qué hace: Contrato canónico de resolución de entity_prefix por tipo de entidad — único SSOT, cierra DT-014. Ningún componente debe hardcodear un prefijo; si falta en el Registry, falla explícito (nunca default silencioso).

Quién lo consume: generate_entity_index_v2.py, lazy_loader.py.

Por qué te sirve saberlo: si ves un error de "prefijo ausente en Registry" en vez de un ID mal formado silencioso, es este contrato funcionando como debe — es una falla intencional, no un bug.

vantage_id_rules.py Qué hace: Módulo único de reglas de detección DEF/REF/heading/boundary para todo el ecosistema de IDs PREFIX:KEY — consolida lógica que antes vivía duplicada (y desincronizada) en 4 scripts distintos (generate_census.py, generate_id_inventory.py, apply_hyperlinks_notion.py, normalize_heading_ids.py).

Por qué te sirve saberlo: antes de esta consolidación (sesión 2026-07-25), cada script tenía su propia noción de "qué es una definición válida" — eso producía falsos huérfanos en el census. Si ves comportamiento inconsistente entre esos 4 scripts hoy, sería regresión de este contrato compartido.

——22.1b MANUAL:SCRIPT-GLOSSARY-L1-TOOLS

Utilidades y Herramientas de Sesión

agent_api.py Qué hace: Capa de consulta en lenguaje natural sobre el índice de entidades — es el motor real detrás de vantage.py ask.

Uso: python3 agent_api.py "texto de consulta" — un solo argumento posicional, entre comillas.

Caso de uso: Ejemplos reales soportados: 'show active roles', 'show archived history', 'show bugs', 'find candidates', 'compare TRACKER:H_xxx TRACKER:H_yyy'.

clean_caches.py (y su wrapper Raycast clean-caches-raycast.sh) Qué hace: Limpieza de cachés de aplicaciones en Mac (Chrome, Safari, Firefox, Edge, y otras) — no toca sesión/login ni LocalStorage, solo caché regenerable. Reporta espacio liberado por ruta.

Uso: Sin flags — se corre directo.

Caso de uso: Tu Mac se siente lento o con poco espacio y quieres liberar caché de navegadores sin riesgo de cerrar sesiones activas.

pipeline_recovery.py Qué hace: Manejo de fallos y "resume operations" del pipeline — guarda checkpoints (save_checkpoint)

para poder retomar una corrida de L1 que se interrumpió a medias.
Uso: Requiere NOTION_TOKEN.

Caso de uso: Si layer_1_run.py se cae a la mitad de un batch grande (ej. por rate limit de Notion), este módulo es el que permite retomar desde el checkpoint en vez de reprocesar todo desde cero.

profile_evolution.py Qué hace: Maneja cambios en la configuración/ perfil del sistema (config/profile_config.yaml) — crea config default si no existe, actualiza progresión de perfil.

Uso: python3 profile_evolution.py — sin flags, interactivo, requiere pyyaml instalado.

Caso de uso: Cuando cambia tu rol objetivo o etapa de carrera (ej. de "Coordinador" a "Dirección") y quieres que el sistema actualice su configuración de perfil de forma guiada en vez de editar el YAML a mano.

weekly_prompt_assembler.py Qué hace: Arma los prompts semanales para los motores externos (L2) — trae Prompt A + Wrappers + Prompt E desde Notion, sustituye la fecha del día, concatena (A + Wrapper) y escribe archivos .md listos para pegar en Gemini/Grok/Perplexity/You.com.

Uso: Sin flags — se corre directo, genera archivos con fecha en el nombre (Prompt_{motor}_{fecha}.md).

Caso de uso: Es el primer paso de tu ciclo semanal de L2 — antes de ir a copiar/pegar prompts a mano en cada motor externo, esto te los pre-arma con la fecha correcta ya sustituida.

patch_vsync_doc.py Qué hace: Patcher de un solo uso (ya ejecutado) que separó la entrada cheat_sheet de vsync_doc.py en dos entradas independientes (alias y change_log) — mismo patrón que patch_new_scripts.py (backup automático + validación de sintaxis antes de escribir).

Uso: python3 patch_vsync_doc.py — sin flags. Es idempotente por diseño de patcher (aunque no verifiqué si tiene el mismo guard explícito).

Nota operativa: Como es un patcher de una sola aplicación histórica (ya corrido, ver ALIASES/CHANGE_LOG separados en tu vsync_doc.py actual), no debería necesitar correrse de nuevo. patch_vsync_doc.py y patch_new_scripts.py fueron MOVIDOS a Archive/Legacy_Scripts/ (saneamiento v9.21.x) — --new-scripts ya no debería detectarlos como pendientes.

⚠ Hallazgo real — extract_score_distribution.py: este script parece ser un borrador abandonado, no una herramienta funcional.

El propio código trae comentarios como "Simulación: voy a asumir que necesito procesar los datos reales" y "Por ahora, voy a mostrar el formato de análisis esperado" — usa datos de muestra hardcoded (sample_data), no consulta Notion. extract_score_distribution.py parece ser una versión temprana/incompleta de extract_scores.py (que sí funciona). Documentado como hallazgo, no corregido — decide tú si vale la pena eliminarlo del árbol para que deje de aparecer en cada gap report. — MOVIDO a Archive/Legacy_Scripts/ (saneamiento v9.21.x)

22.3 MANUAL:SCRIPT-GLOSSARY-L4Layer 4 — Version Control & Sync Documental

vdoc.pyQué hace: Wrapper de orquestación — decide dirección de sync (Notion↔local) y dispara vsync_doc.py + git_sync.py.

Uso (tokens posicionales, no flags tradicionales):

Token	Caso de uso
dry	Preview de lo que haría en modo auto, sin escribir nada. Úsalo antes de cualquier sync si no estás seguro de qué documento cambió más recientemente.
notion	Fuerzas Notion→local (pide confirmación). Úsalo si sabes que editaste en Notion y el local está desactualizado.
local	Fuerzas local→Notion (PATCH puntual). ⚠ Nota real: pese a que el resto del sistema pide confirmación para direcciones forzadas, local tiene una excepción temporal en el código y NO pide confirmación — ejecuta directo.
auto	Deja que el script decida por hash cuál lado está más reciente.
<documento> (ej. kernel, brief)	Restringe el sync a un solo documento en vez de los 7.
vsync_doc.py	Qué hace: Motor real de sincronización documento-por-documento (invocado por vdoc.py, no directo normalmente).

Flags:	
Flag	Caso de uso
---	---
--direction {notion,auto,local}	Igual lógica que los tokens de vdoc.py, pero si necesitas invocar el motor directo (debugging).
--dry-run	Preview sin aplicar ni auto-commit.
--doc {kernel,system_prompt,career_cano n,manual,alias,change_log,brief}	Nota real: maneja 7 documentos, incluyendo brief — aunque la documentación textual del Manual describe un catálogo de 6.
git_sync.pyQué hace: Genera commit y push del árbol VANTAGE hacia GitHub.	
Flags:	
Flag	Caso de uso
---	---
--dry	Antes de un push automático, revisa qué archivos entrarían al commit sin ejecutar nada.
vsum.pyQué hace: Genera resúmenes de archivos Markdown/ texto vía Groq o Gemini.	
Flags:	
Flag	Caso de uso
---	---
file (posicional, requerido)	Ruta al archivo local a resumir. ⚠ Nota real: pese a lo que sugiere cualquier doc sobre "Claude Share URLs", el código solo acepta rutas de archivo local — una URL fallará.
-o/--output <ruta>	Si quieres guardar el resumen en vez de verlo en pantalla — útil para encadenar con otro proceso.

<code>-m/--model {groq,gemini}</code>	Si Groq está teniendo rate limits, cambia manualmente a Gemini con este flag en vez de esperar el fallback automático.
<code>--notion</code>	⚠ Flag vestigial — se parsea pero no tiene ningún efecto en el código actual. No lo uses esperando que cree una página en Notion.

——22.4 MANUAL:SCRIPT-GLOSSARY-DASHBOARD

Dashboard — Servidor Local de Visualización

dashboard_start.sh → dashboard_server.pyQué hace: Levanta el servidor local del Dashboard, corre smoke test, abre el navegador.

Uso: Sin flags — un solo comando hace todo el ciclo (start → healthcheck → smoke test → abrir browser). Si el smoke test falla, no abre el navegador y te avisa por notificación de sistema.

layer_1_run_dash.pyQué hace: Variante de layer_1_run.py adaptada para ser invocada desde el Dashboard web en vez de Terminal.

Uso: Sin flags CLI propios — se invoca vía las rutas HTTP del Dashboard, no directo.

22.4a MANUAL:SCRIPT-GLOSSARY-DASHBOARD-MODULES

Módulos Internos del Dashboard (sin CLI — arquitectura, no scripts ejecutables)

Al igual que los módulos de 22.1a, estos 5 no tienen flags porque no se ejecutan como scripts sueltos — son la arquitectura interna del servidor Flask del Dashboard. Documentados por su rol, no por CLI.

| Módulo | Rol |

| --- | --- |

| dashboard_config.py | Carga config/dashboard.env, expone NOTION_TOKEN, DATABASE_ID (desde NOTION_DB_OPPORTUNITIES) y DB_PATH (SQLite local del Dashboard) al resto de módulos. |

| dashboard_db.py | Capa de acceso a la base SQLite local (dashboard_instances.db) — inicialización de schema y operaciones CRUD internas del Dashboard (no confundir con el Tracker de Notion). |

| dashboard_notion.py | Cliente Notion del Dashboard — reutiliza NOTION_TOKEN/DATABASE_ID de dashboard_config.py, importa txt() de layer_1_run.py para parseo de propiedades. |

| dashboard_routes.py | Rutas HTTP Flask (jsonify/request) — endpoints del Dashboard, consume dashboard_notion.py para las queries reales (ej. query_blocked_vacancies, write_patch_to_notion). |

| dashboard_validation.py | Reutiliza validación core del pipeline (validate_url_pre_ingestion, calculate_score_v6, get_vm_scope, get_role_class, gate) directamente de layer_1_run.py — garantiza que el Dashboard aplique

exactamente la misma lógica de scoring/gate que el pipeline CLI, no una copia paralela. |

smoke_dashboard.py Qué hace: Smoke test del servidor Dashboard corriendo en <http://127.0.0.1:8000> — verifica /health y /blocked-vacancies, imprime SMOKE PASSED/SMOKE FAILED.

Uso: Sin flags — requiere que el Dashboard ya esté corriendo localmente (lo invoca dashboard_start.sh como parte de su ciclo de arranque).

Caso de uso: Si el Dashboard se ve raro tras un cambio de código, corre esto antes de asumir que es un bug visual — confirma si el backend responde correctamente primero.

——22.5 MANUAL:SCRIPT-GLOSSARY-RAYCAST

Raycast — Atajos de Un Click

Estos scripts son wrappers delgados — su único trabajo es invocar el script real de la capa correspondiente con notificaciones de sistema (sonido de éxito/error). No tienen flags propios más allá de lo que reenvían.

| Wrapper Raycast | Invoca | Nota operativa |

| --- | --- | --- |

| vantage-health.sh | health_check.py | Sin flags. |

| vantage-vl1.sh → layer_1_wrapper.sh | layer_1_pipeline.sh | Si no le pasas argumentos, auto-detecta el feed JSON más reciente en Layer_1/feeds/. |

| vantage-vl3.sh → layer_3_mail.sh | layer_3_mail.py | Sin flags — valida que exista .venv y config/layer_3.env antes de correr. |

| vantage-dedup.sh / vantage-opport-dedup.sh | dedup_opportunities.py (sin args) |  No expone --clear — para limpiar un flag puntual necesitas Terminal directo. |

| vantage-vgit.sh → git_sync_wrapper.sh | git_sync.py | Reenvía "\$@" — sí puedes pasarle --dry vía Raycast si tu atajo lo permite. Loguea cada corrida en /tmp/vantage_l4_gitsync.log. |

| vantage-vdoc-dry.sh | vdoc.py dry | Atajo directo al modo preview. |

| vantage-vdoc-notion.sh | vdoc.py notion | Atajo directo al modo forzado Notion→local (pide confirmación). |

| vantage-vd.sh | vdoc.py (modo según config del atajo) | Revisa el contenido del script si necesitas saber qué dirección dispara por default. |

| vantage-census.sh | generate_census.py | Sin --debug-id expuesto — census completo únicamente. |

| vantage-versions-bootstrap.sh | verify_versions.py --bootstrap | |

| vantage-versions-sync.sh | verify_versions.py --sync | |

| vantage-versions-scripts-gap.sh | verify_versions.py --scripts | |

| vantage-hyperlinks-dry.sh | apply_hyperlinks_notion.py --all (sin --apply) | |

| vantage-hyperlinks-apply.sh | apply_hyperlinks_notion.py --all --apply | 

Escritura real de un solo click — sin gate de confirmación adicional del lado Raycast. |

| vantage-status.sh | status_report.py | Sin flags. |

| vantage-sync.sh | (revisar contenido — probablemente alias de vdoc.py auto o

vsync_doc.py) | Pendiente de confirmación directa si se usa activamente. |
| clean-caches-raycast.sh | clean_caches.sh | Limpieza de cachés de apps
(Chrome, Figma, Notion, etc.) — no toca sesión/login, solo caché regenerable. |
——**22.6 MANUAL:SCRIPT-GLOSSARY-XREF**Matriz de Transición de Estados —
Ciclo de Vida del Script
Vista tabular consolidada, mismo patrón que [KERNEL:GATE-DECISION-011]
(Matriz de Transición de vacantes) — aplicada aquí al ciclo de vida de un script
dentro del sistema de documentación (disco → Glosario → Script Library).
Referencia canónica para el skill vantage-sync-script-glossary (punto E) y para
verify_versions.py --new-scripts (punto D) — no reemplaza la descripción en
prosa de cada script, la complementa con indexación de estados.

Estado Origen	Evento / Trigger	Guard / Regla	Estado Destino	Componen te	Efecto
[ENTRY]	Commit de archivo .py /.sh nuevo en árbol activo (Layer_1/3/4, Dashboard , Raycast)	No excluido por EXCLUDE D_DIR_NAMES/ EXCLUDE D_FILE_PREFIXES	NO_DOCUMENTADO	Filesystem (git)	Ninguno — script existe, cero registro
NO_DOCUMENTADO	Operador corre vversions --new-scripts	Nombre de archivo ausente en SCRIPT_GLOSSARY_PATH (local, sin Notion)	DETECTADO	Python (local, read-only)	Exit code 1 — reporte en stdout, sin escritura

DETECTADO	Claude invoca vantage-sync-script-glossary	Lectura del script fuente (grep de flags/env vars) + DRY RUN de ambas entradas propuestas	PROPUESTO	Claude (AI Component)	Ninguno aún — solo preview, sin escritura
PROPUESTO	Operador confirma APROBAR _WRITE	Escritura dual atómica: entrada en Glosario (Notion, página Manual) + fila en Script Library (Notion DB)	DOCUMENTADO	Claude + Notion MCP	Página Manual actualizada; fila Script Library creada; write-back verification en ambos
PROPUESTO	Operador rechaza o no confirma	Sin APROBAR _WRITE	DETECTADO	Humano	Sin cambio — permanece pendiente para próxima corrida

DOCUMENTADO	Edición posterior del script (nuevo flag agregado)	Diff entre flags reales (grep) y flags documentados en Glosario	DESACTUALIZADO	Python + Claude (detección manual o futura extensión de --new-scripts)	Marcado  en Glosario — no auto-correctado, requiere revisión igual que hallazgos de auditoría (ver lista al final de esta sección)
DOCUMENTADO	Script renombrado con prefijo DEPRECATED_ o movido a carpeta excluida	EXCLUDE_D_FILE_PREFIXES / EXCLUDE_D_DIR_NAMES aplica en el próximo scan_committed_assets()	HUÉRFANO_GLOSARIO	Filesystem (git) + Python (detección pasiva)	Ninguno automático — entrada en Glosario/Script Library queda obsoleta, sin flag de alerta hasta auditoría manual
DOCUMENTADO (Script Library)	Fila marcada "Activo" en Notion sin archivo correspondiente en disco	vversions --scripts (ya existente, vía Notion)	HUÉRFANO_NOTION	Python (read-only) + Notion	Reportado en gap report — remediación manual, mismo patrón que hoy

Reglas de mantenimiento derivadas de la matriz:					
---	--	--	--	--	--

- Todo script nuevo agregado al árbol activo transita [ENTRY] → NO_DOCUMENTADO → DETECTADO → PROPUESTO → DOCUMENTADO
 - nunca salta directo a DOCUMENTADO sin pasar por DRY RUN + APROBAR_WRITE (mismo invariante que [KERNEL:DATA-FLOW]).
 - El estado DOCUMENTADO es doble — requiere presencia simultánea en Glosario y Script Library. Un script en solo uno de los dos no es DOCUMENTADO, es un estado intermedio no listado arriba porque hoy no debería poder ocurrir (la escritura de vantage-sync-script-glossary es atómica en ambos destinos).
 - DESACTUALIZADO y HUÉRFANO_* no tienen remediación automática por diseño — son señales para auditoría del operador, igual que los hallazgos  ya documentados en este Glosario. Documentar la discrepancia, no "arreglarla" sin instrucción explícita.
 - Esta matriz es la fuente de verdad para el diseño del skill vantage-sync-script-glossary — cualquier cambio a su lógica de transición pasa por vantage-documentacion-transversal-propuesta sobre este nodo (MANUAL:SCRIPT-GLOSSARY-XREF), no por edición directa del skill.
 - Hallazgos de discrepancia activos (heredados de auditoría arena.ia, verificados contra código fuente):
- 1. layer_1_pipeline.sh batch no reenvía -execute a batch_operations.py — siempre corre en modo definido por el propio script.
- 2. vdoc.py local no pide confirmación pese a que el resto de direcciones forzadas sí (excepción temporal marcada en el propio código).
- 3. vsync_doc.py --doc maneja 7 documentos (incluye brief), no 6.
- 4. dedup_opportunities.py --clear requiere posición fija en sys.argv, no es un flag argparse real.
- 5. vsum.py --notion se parsea pero no tiene efecto — vestigial.
- 6. cross_tracker_match.py --dry-run no puede desactivarse — default True sin opuesto.
- 7. GROQ_MAX_EMAILS_PER_RUN: Manual/Aliases citan valores distintos entre sí; código usa 10. — RESUELTO (v9.21.x, saneamiento estructural): Manual §12 alineado a 10 correos.

——23 MANUAL:SKILL-GLOSSARY

Glosario de Skills — Referencia Operativa en Humano

Propósito: Traducir cada skill de Claude (/mnt/skills/user/) a lenguaje operativo — qué hace, qué la dispara, si requiere gate de escritura, y su convención de

anuncio. Contraparte de MANUAL:SCRIPT-GLOSSARY (§22), aplicada a skills en vez de scripts. La Skill Library (Notion DB) es su índice corto y estructurado; se referencian entre sí.

Desde 2026-08-17, este glosario también respalda el consumo multi-agente del manifiesto skills/triggers.json (ver SP:BOOTLOADER-001, KERNEL:ARCHITECTURE-L4) — cada fila de Skill Library es la fuente de notion_id para agentes con MCP Notion.

——23.1 MANUAL:SKILL-GLOSSARY-COREPipeline CV y Ciclo de Sesión

Skill	Propósito	Trigger	Gate	Anuncio
vantage-session-open	Bootstrap de sesión (Ledger→Contexto)	Inicio de sesión / invocación explícita	✗ (housekeeping, exento de APROBAR_WRITE)	SESSION-OPENING... / SESSION-OPENED
vantage-session-close	Cierre de sesión (Changelog+Ledger+Sync)	Fin de sesión / invocación explícita	✓	CLOSING SESSION... / SESSION CLOSED
vantage-cv-a	Analiza JD/URL, produce HANDOFF con Positioning Mode	CV-A [URL/JD]	✗	No especificado (gap, ver 23.5)
vantage-cv-b	Construye CV final con Figma Tags desde HANDOFF	CV-B [HANDOFF]	✓	No especificado (gap, ver 23.5)
vantage-qa	Audita PDF de CV final, veredicto GO/NO-GO	QA [PDF]	✗	No especificado (gap, ver 23.5)
vantage-present-handoff	Handoff compacto de sesión para continuidad	Fin de sesión / cambio de chat	✗	HANDING OFF... / HANDOFF DELIVERED

——23.2 MANUAL:SKILL-GLOSSARY-HOUSEKEEPINGSincronización y Mantenimiento Documental

Skill	Propósito	Trigger	Gate	Anuncio
vantage-sync-script-library	Sincroniza Script Library Notion vs disco	"sincronizar Script Library" / gap report	✓	SYNCING SCRIPT LIBRARY... / SCRIPT LIBRARY SYNCED
vantage-sync-skill-library	Sincroniza Skill Library Notion vs disco .skill	"sincronizar Skill Library" / gap report	✓	SYNCING SKILL LIBRARY... / SKILL LIBRARY SYNCED
vantage-sync-assets	Orquesta las 6 sync de Libraries/ Glossaries/ Census/ Hyperlinks en orden fijo	"sync assets" / "sincronizar assets"	✓	SYNCING ASSETS... / ASSETS SYNCED
vantage-sync-script-glossary	Sincroniza apéndice §22 vs disco	"sincronizar Script Glossary" / gap --new-scripts	✓	No especificado (gap, ver 23.5)
vantage-sync-census-spec	Da de alta IDs huérfanos en CENSUS_SPEC	"sincronizar Census Spec" / huérfanos en vcensus	✓	SYNCING CENSUS SPEC... / CENSUS SPEC SYNCED
vantage-hyperlink-loop	Ciclo Census→Hyperlinks→Sync	Housekeeping de navegación / invocación explícita	✓	REGENERATING NAVIGATION LOOP... / NAVIGATION LOOP FINISHED

vantage-housekeeping-tracker	Orquesta housekeeping de trackers (Bug/Task → VANTAGE Tracker → Change Log) en orden fijo	"housekeeping trackers" / "tidy trackers"	✓	HOUSEKEEPING TRACKERS... / TRACKERS HOUSEKEPT
vantage-housekeeping-archive	Consolida detección→ marcado→verificación de candidatos a archivar (Dedup_Flag/ Next_Action) en un solo procedimiento, absorbiendo el ciclo antes cubierto por escaneo visual (ver KERNEL:GATE-DECISION-007)	"housekeeping archive" / candidatos detectados vía Dedup_Flag/ Next_Action	✓	ARCHIVING HOUSEKEEPING... / ARCHIVE HOUSEKEPT
vantage-create-bug-task	Crea ticket en Bug/Task Tracker	Reporte de defecto o tarea pendiente	✓	LOGGING TICKET... / TICKET LOGGED
vantage-tidy-bug-task-tracker	Marca tickets resueltos como Archivar=True	Resolución directa o vía Change Log	✓	TIDYING TRACKER... / TRACKER TIDIED

vantage-tidy-opportunities-tracker	Marca duplicados/ expiradas en VANTAGE Tracker — no redacta la causa (ver KERNEL:GATE-DECISION-013): opera sobre registros donde VL1 ya documentó la razón	Housekeeping de vacantes	✓	TIDYING OPPORTUNITIES... / OPPORTUNITIES TIDIED
vantage-tidy-changelog	Recorta Change Log a últimas 10 entradas	Exceso >10 entradas / housekeeping	✓	TIDYING CHANGELOG ... / CHANGELOG TIDIED
vantage-documentation-transversal-propuesta	Mapea nodos fundacionales afectados, sin escribir	"propuesta de documentación transversal" / gap estructural	✗	BEGINNING DOCUMENTATION MAPPING... / cierre no especificado (gap, ver 23.5)
vantage-documentation-transversal-implementación	Ejecuta escritura de documentación ya autorizada	Post-APROBAR_W RITE de propuesta	✓	RESUMING DOCUMENTATION — IMPLEMENTATION PHASE... / DOCUMENTATION FINISHED

—— **23.3 MANUAL:SKILL-GLOSSARY-AUDIT** Auditoría y Continuidad

Skill	Propósito	Trigger	Gate	Anuncio
-------	-----------	---------	------	---------

vantage-audit-navigation-brief	[DEPRECATE D] Funcionalidad integrada en vantage-documentación-transversal-propuesta	N/A — usar vantage-documentación-transversal-propuesta	✗ (solo lectura, nunca escribe Notion)	No aplica (deprecada)
extract-learnings	[DEPRECATE D] Actividad post-mortem esporádica, no skill operativa recurrente	Post-fumble evitable o creación de skill nueva	✗	No aplica (housekeeping interno)
vantage-skill-updater	Evalúa compliance de skills contra KERNEL requirements , propone actualizaciones	"actualizar skills con requisitos VANTAGE" / "evaluar compliance de skills"	✓	BEGINNING SKILL EVALUATION ... / SKILL EVALUATION COMPLETE

—— **23.4 MANUAL:SKILL-GLOSSARY-STYLE** Estilos de Escritura y Generación (activación por invocación explícita, no por trigger operativo)

Skill	Propósito	Trigger
critico-pensante-style	Reflexivo/analítico, cuestiona contradicciones	Invocación exacta por nombre
retail-auditor-style	Auditoría sistemática de VM/retail multi-tienda	Invocación exacta por nombre
socio-estrategico-style	Directo/eficiente para VM, sin rodeos	Invocación exacta por nombre
corporate-communication-style	Cartas de recomendación formales	Invocación exacta por nombre

prompt-master	Genera prompts optimizados para otras IAs	Pedido explícito de prompt para IA/Cursor/Midjourney/etc.
tailored-resume-generator	Genera CVs adaptados a una vacante específica	Solicitud de CV a medida + JD

——23.5 MANUAL:SKILL-GLOSSARY-XREF

Gaps Abiertos (hallazgos, no corregidos en esta pasada)

- Capa null en 24/25 filas de Skill Library (Notion) — campo definido en schema, prácticamente sin uso.
 - Anuncio no especificado en 5 skills operativas que deberían tenerlo por KERNEL:DOCUMENTATION-005:
 - vantage-cv-a, vantage-cv-b, vantage-qa, vantage-sync-script-glossary, y el cierre de vantage-documentacion-transversal-propuesta.